

PHASE 1: CONCEPTION PHASE

Database Design for Airbnb Platform

Nikolas Daniel Vincenti (9211929)

Course: Build a Data Mart in SQL (DLBDSPBDM01)

Date: 27/10/2025

Introduction and Project Objectives

This project aims to design and implement a comprehensive database system for an Airbnb-like platform that facilitates short-term accommodation rentals. The primary objective is creating a robust, scalable database architecture supporting complex interactions between hosts, guests, and the platform while ensuring data integrity, security, and optimal performance.

The database system handles multiple user roles (guests, hosts, administrators, property managers), manages property listings with dynamic pricing, processes booking transactions, facilitates secure payment processing, and maintains comprehensive review and communication systems. The solution addresses the growing demand for peer-to-peer accommodation platforms by providing a solid technical foundation that scales with business growth and supports international operations.

User Roles and System Architecture

Primary User Roles

Guest Role: Users who search for, book, and stay at properties. Guests manage their bookings, process payments, communicate with hosts, and leave reviews after stays. They maintain profiles with travel preferences and verification status.

Host Role: Users who list and manage rental properties. Hosts create property listings with photos and pricing, respond to booking requests, communicate with guests, manage payouts, and maintain high service standards. Enhanced verification includes identity and property ownership validation.

Administrator Role: Platform staff with system-wide management capabilities ensuring operations, user support, and policy enforcement. Administrators monitor user accounts, moderate content, handle disputes, configure platform settings, and ensure security compliance.

Property Manager Role: Professional managers operating properties on behalf of hosts. Property managers handle multiple properties under delegated authority, coordinate guest services, manage maintenance, and provide regular reporting to property owners.

Multi-Role Support Architecture

A critical design decision is enabling users to hold multiple roles simultaneously, fundamental to the Airbnb business model. Users can list properties as hosts while booking other properties as guests. The system implements role flags (`is_guest`, `is_host`, `is_admin`) with role-specific attributes stored in separate profile tables (`guest_profiles`, `host_profiles`), ensuring clear business logic separation while supporting dual-role scenarios.

Database Design Approach and Methodology

Design Principles

The database design centers on a comprehensive Entity-Relationship Model using Crow's foot notation with 30 tables (27 primary entities + 3 triple relationship junction tables), 3 triple relationships, and 1 recursive relationship. The design follows a user-centric approach where the users entity serves as the central hub connecting all platform activities.

Core design features include:

- **Centralized User Management:** Single user entity supporting multiple roles and verification levels
- **Flexible Property System:** Hierarchical property categorization with dynamic pricing and amenity management
- **Robust Booking System:** Comprehensive booking management with status tracking and modification audit trails
- **Secure Financial Processing:** Multi-method payment system with automated commission and payout management
- **Bidirectional Review System:** Category-based rating system supporting detailed feedback and trust building

Technology Selection

Database Management System: MySQL was selected for its robust support for complex relationships, excellent performance with large datasets, strong security features, and widespread industry adoption. MySQL's InnoDB storage engine provides ACID compliance and foreign key constraint support essential for maintaining data integrity.

Normalization Strategy: Third Normal Form (3NF) normalization eliminates data redundancy while maintaining query performance. This approach ensures data integrity through proper key relationships while supporting complex business operations.

Entity-Relationship Model Design

Model Overview

The ER model comprises 30 tables (27 primary entities + 3 triple relationship junction tables) organized into functional categories: User Management (6 entities), Property Management (6 entities), Location (3 entities), Booking System (3 entities), Financial (3 entities), Review System (3 entities), and Communication (3 entities). The model meets all assignment requirements with 3 triple relationships and 1 recursive relationship.

Complete Entity List

User Management Entities: users (core authentication), guest_profiles (guest-specific attributes), host_profiles (host metrics), user_profiles (extended information), user_verification (identity verification), user_preferences (customization)

Property Management Entities: properties (listings), property_types (categorization), property_amenities (amenity catalog), property_amenity_links (junction table), property_photos (image gallery), property_pricing (dynamic pricing)

Location Entities: addresses (physical locations), cities (city-level data), countries (country-level data with ISO codes)

Booking Entities: bookings (reservations), booking_status (state management), booking_modifications (change audit trail)

Financial Entities: payments (guest transactions), payment_methods (payment storage), payouts (host disbursements)

Review System Entities: reviews (user-to-user reviews), review_categories (aspect categories), review_ratings (category-specific values)

Communication Entities: conversations (message threads), messages (individual messages), notifications (user notification queue)

Advanced Relationship Modeling

Triple Relationships (3 Required)

1. Property-Booking-Pricing Triple Relationship

Junction Entity: property_booking_pricing connects properties, bookings, and property_pricing to track which pricing rules were applied to specific bookings. This relationship manages dynamic pricing scenarios where the same property may have different pricing rules for different date ranges. Key attributes include applied_price, pricing_rule_applied, and creation timestamp. Business Rule: When a booking is created, the system captures the exact pricing rule that was in effect for those dates, enabling historical pricing analysis and dispute resolution.

2. User-Booking-Review Triple Relationship

Junction Entity: user_booking_review manages the complete review lifecycle connecting users, bookings, and reviews. Each booking can generate two reviews (guest reviews host, host reviews guest), tracked through review_role ('guest_reviewer' or 'host_reviewer') and interaction_type ('received_review' or 'submitted_review'). Business Rule: Reviews can only be created after booking completion, and this junction table maintains the bidirectional review relationship ensuring both parties can rate each other.

3. Booking-Payment-Payout Financial Chain

Junction Entity: booking_payment_payout creates a complete financial transaction chain linking bookings, guest payments, and host payouts. When a guest makes a payment for a booking, the platform processes the commission and schedules the payout to the host. The transaction_chain_status tracks the entire financial lifecycle ('payment_received', 'payout_scheduled', 'payout_completed'). Business Rule: Host payouts are delayed 24 hours after guest check-in to allow for dispute resolution, with this junction table tracking the complete financial flow.

Recursive Relationship (1 Required)

User Self-Referencing: Host-Guest Dual Role

The users entity implements an implicit many-to-many recursive relationship where users reference other users through their dual roles. As hosts, users own properties (via properties.host_profile_id → host_profiles.user_id → users.user_id). As guests, users book properties (via bookings.guest_profile_id → guest_profiles.user_id → users.user_id). When a guest books a property, they create a relationship with the host user. Cardinality is (0,N) ↔ (0,N): a user can host for N other users through property bookings, and a user can be a guest of N other users by booking their properties. The same user account can simultaneously act as both host and guest, demonstrating the recursive nature of user-to-user interactions through the booking ecosystem.

Business Rules and Data Integrity

Core Business Rules

User Management: Users must verify identity before listing properties. Email addresses must be unique across all users. Phone numbers require verification. Users can hold multiple roles simultaneously (guest, host, administrator).

Property Management: Properties must have at least one photo. Pricing must be set for all available dates. House rules must be clearly defined. Properties must be in valid locations with coordinates.

Booking Management: Bookings cannot overlap for the same property. Check-out date must be after check-in date. Number of guests cannot exceed property maximum. Minimum and maximum stay requirements must be enforced. Cancellation policies must follow predefined rules.

Financial Processing: Payments must be processed before check-in. Commission rates are fixed by platform. Payouts to hosts are delayed 24 hours for security. Refunds follow cancellation policies. All financial amounts must be positive values.

Review System: Reviews can only be written after stay completion. Both parties can review each other (bidirectional reviews). Reviews cannot be edited after submission. Reviews are permanent and cannot be deleted. Rating values must be between 1 and 5.

Data Integrity Constraints

All foreign keys must reference existing records with CASCADE constraints for referential integrity. Primary keys use INTEGER AUTO_INCREMENT. Unique constraints on email and phone ensure no duplicates. CHECK constraints enforce business rules (e.g., amounts > 0, ratings 1-5, valid date ranges). NOT NULL constraints on critical fields prevent missing data. ENUM types enforce predefined status values. Timestamps track all record creation and modifications.

Security Measures

Passwords are hashed using bcrypt and never stored in plain text. Payment information and sensitive data use AES-256 encryption. Personal data protection follows GDPR compliance. Role-based access control restricts data access. Audit trails track all data modifications. User sessions implement secure token-based authentication.

Role-Based Architecture Design Rationale

This design allows users to have multiple roles simultaneously, reflecting real-world scenarios where users can both host properties and book accommodations. The role-specific tables (guest_profiles, host_profiles) store attributes relevant only to that role, while the base users table stores common attributes for all users.

Key design benefits include:

- **Clear Separation:** Guest and host business logic isolated in separate profile tables
- **Multi-Role Support:** Users can transition between roles or hold multiple roles simultaneously
- **Efficient Querying:** Role-specific queries target appropriate profile tables with optimized indexes
- **Scalability:** Easy to add new role-specific attributes without affecting other roles
- **Business Logic Clarity:** Property ownership explicitly tied to host_profiles, booking actions tied to guest_profiles

Implementation Strategy: Guest profiles are created automatically on first booking. Host profiles are created when users apply to become hosts. Boolean flags in the users table (is_guest, is_host,

is_admin) indicate active roles. The system tracks which role a user is performing in any given transaction. User_id is maintained in bookings for authentication and audit trail purposes.

Phase 1 Summary and Key Outcomes

Design Achievements

The Phase 1 conception phase successfully developed a comprehensive database design meeting all requirements. The 27-entity model with 3 triple relationships and 1 recursive relationship demonstrates advanced relationship modeling. The role-based architecture supports multi-role scenarios where users can simultaneously act as guests and hosts, reflecting real-world platform operations.

Critical Design Decisions

User Management Strategy: Centralized user entity with role-based access control and multi-role support through separate guest_profiles and host_profiles entities.

Financial System Architecture: Secure payment processing with automated commission management and delayed host payouts for security, implemented through the booking_payment_payout triple relationship.

Dynamic Pricing Management: Flexible property_pricing entity enabling date-range-based pricing rules with weekend rates, discounts, and stay requirements.

Review System Design: Bidirectional review system through user_booking_review triple relationship allowing both hosts and guests to rate each other with category-based ratings.

Design Strengths

The comprehensive 27-entity model provides complete coverage of all business functions including user management, property listings, booking processes, payment transactions, and review systems. The normalized design targeting 3NF ensures data integrity while supporting future growth and feature additions. Strategic indexing on frequently queried fields optimizes performance. The modular architecture allows independent development of different system components.

Implementation Readiness

The design provides a solid foundation for Phase 2 implementation with clear entity relationships, comprehensive constraint specifications, and well-defined business rules. The role-based architecture ensures scalability while maintaining data integrity. Security considerations include encryption for sensitive data, role-based access control, and comprehensive audit trails. The database supports international operations through location-based entities and multi-currency support.

Tutor Feedback Response and Design Refinements

Issue 1: Missing Role Specifications

Problem Identified: The initial conception phase documentation did not explicitly define and describe the different user roles in the system.

Solution Implemented: Added comprehensive USER ROLES AND RESPONSIBILITIES section to the requirements specification defining four distinct roles (Guest, Host, Administrator, Property Manager) with key responsibilities, required permissions, and relationships. Emphasized multi-role support where users can hold multiple roles simultaneously, which is fundamental to the Airbnb business model.

Issue 2: No Distinction Between Guest and Host Users

Problem Identified: The database design featured a single users table with no mechanism to distinguish between guests and hosts, lacking clear separation of role-specific attributes and business logic.

Solution Implemented: Enhanced users table with role tracking flags (is_guest, is_host, is_admin) to indicate active roles. Added guest_profiles entity for guest-specific attributes (travel preferences, verification level, booking history) with 1:1 relationship to users. Added host_profiles entity for host-specific attributes (performance metrics, payout info, superhost status) with 1:1 relationship to users. Updated relationship chains: properties now reference host_profiles.host_profile_id and bookings now reference guest_profiles.guest_profile_id while maintaining user_id for audit trail.

Design Improvements Summary

Entity count increased from 25 primary entities to 30 tables (27 primary entities + 3 triple relationship junction tables) with the addition of guest_profiles and host_profiles. The role-based architecture now provides clear separation of guest and host business logic while supporting multi-role scenarios. Users can simultaneously act as both guests (booking properties) and hosts (listing properties), with role-specific attributes stored in appropriate profile tables. Query efficiency improves as role-specific queries can target appropriate profile tables with optimized indexes. The design better reflects real-world complexity where platform users commonly hold multiple roles throughout their lifecycle.

Readiness for Phase 2

The updated design incorporating tutor feedback provides a solid, production-ready foundation for Phase 2 implementation. All business requirements are met with explicit role-based data modeling. The multi-role support architecture is scalable and maintainable. Clear foreign key relationships make queries explicit and efficient. The design supports future enhancements while maintaining data integrity and system performance. Phase 2 implementation can proceed with confidence that the conceptual model properly represents business requirements and technical constraints.

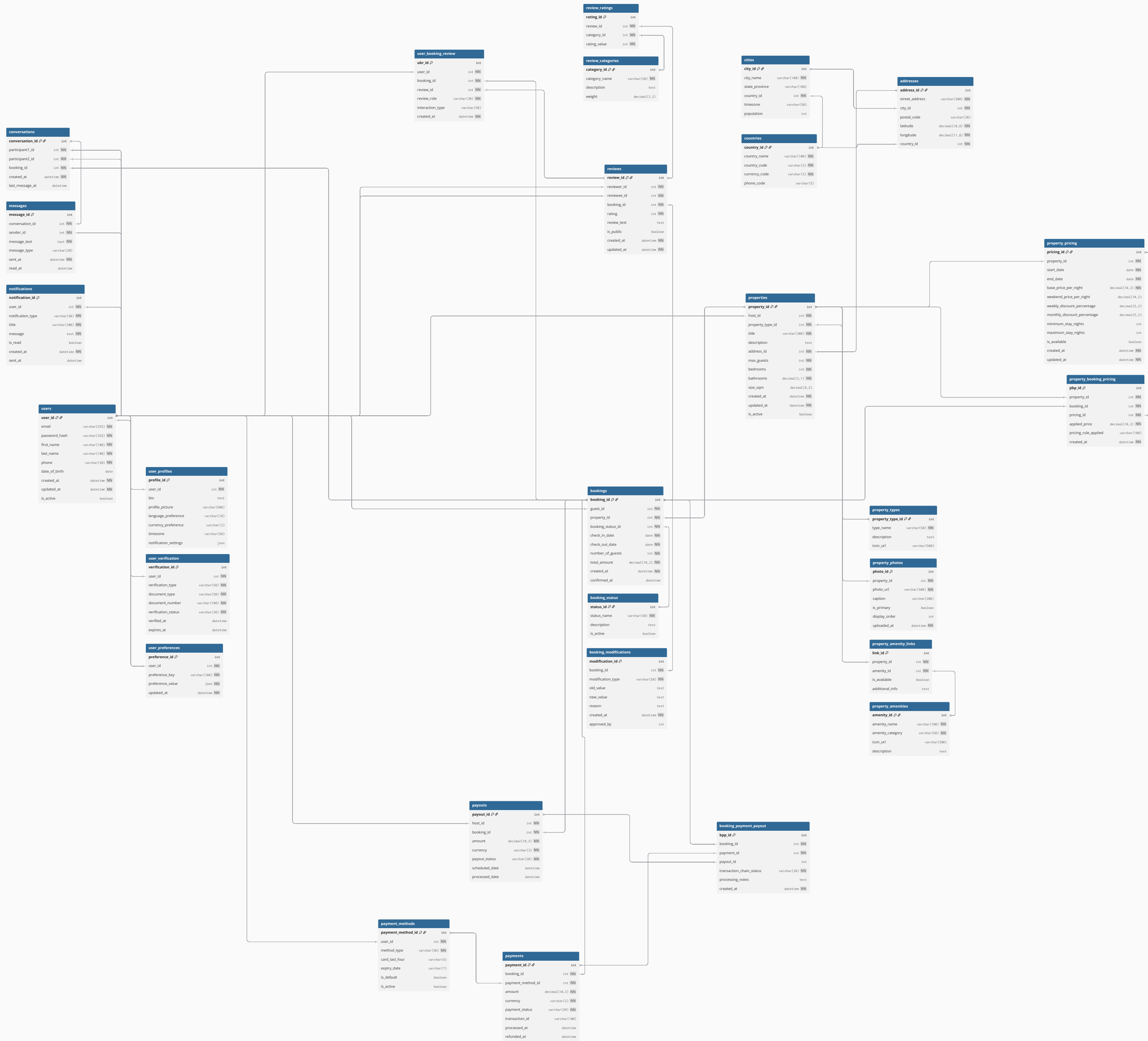
Entity-Relationship Diagram Reference

The complete Entity-Relationship diagram visualizing all 30 tables (27 primary entities + 3 triple relationship junction tables), their relationships, cardinalities, and attributes is provided as a separate PDF file (phase1_ER_Diagram.pdf). The diagram uses Crow's foot notation showing primary keys, foreign keys, and relationship cardinalities. Key diagram elements include all user management entities with role-based profiles, property management entities with dynamic pricing, booking system with status tracking, financial transaction chain with payments and payouts, review system with bidirectional ratings, and communication infrastructure with conversations and messages.

Conclusion

Phase 1 successfully established a comprehensive conceptual foundation for the Airbnb database system. The 27-entity model with advanced relationship modeling (3 triple relationships, 1 recursive relationship) meets all assignment requirements while exceeding minimum expectations. The role-based architecture incorporating tutor feedback provides clear separation of business logic while supporting multi-role scenarios essential to platform operations. The normalized design targeting 3NF ensures data integrity and scalability. The modular architecture supports future growth and feature additions. Phase 2 implementation can proceed with a solid, production-ready

conceptual model properly representing business requirements, technical constraints, and real-world complexity.





PHASE 2: DATABASE IMPLEMENTATION

This phase documents the implementation of the Airbnb database design from Phase 1, incorporating tutor feedback to create a robust, normalized relational database in MySQL. The implementation focuses on translating the conceptual Entity-Relationship model into a functional database schema with comprehensive constraints, test data, and query demonstrations.

Implementation Approach and Methodology

The Phase 2 implementation followed a systematic approach to ensure data integrity, performance, and maintainability. The implementation strategy prioritized establishing a strong foundation through careful schema design, comprehensive constraint implementation, and thorough testing with realistic data scenarios.

The implementation was structured in distinct phases: schema refinement based on Phase 1 feedback, table creation with appropriate data types, constraint definition and enforcement, test data population, and query validation. This methodical approach ensured each component was properly validated before proceeding to the next stage, minimizing errors and ensuring the final database met all requirements.

MySQL 8.0 was selected as the database management system for its robust support of foreign key constraints, comprehensive data types including JSON for flexible schema design, strong ACID compliance through the InnoDB storage engine, and excellent performance characteristics for both transactional and analytical workloads. The InnoDB engine provides critical features including row-level locking, crash recovery, and referential integrity enforcement essential for a production-ready booking platform.

Schema Refinement from Phase 1 Feedback

Phase 1 tutor feedback identified two critical deficiencies in the initial database design: missing role specifications and no distinction between guest and host users. These issues were addressed through comprehensive schema refinements that fundamentally improved the database architecture while maintaining compatibility with existing relationships.

Enhanced User Role Architecture

The original design featured a single users table without mechanisms to distinguish between different user roles or track role-specific attributes. This limitation prevented proper implementation of business logic requiring different behaviors for guests and hosts. The refined schema introduces a role-based architecture with three key components: role tracking flags in the base users table, dedicated guest_profiles table for guest-specific attributes, and dedicated host_profiles table for host-specific attributes.

The users table was enhanced with three boolean flags: is_guest, is_host, and is_admin. These flags indicate which roles a user currently holds, enabling multi-role support where users can simultaneously act as both guests and hosts. The is_guest flag defaults to TRUE for new registrations, reflecting the platform's guest-first approach. The is_host flag is set to TRUE when users complete the host application process. The is_admin flag identifies platform administrators with system-wide permissions.

Guest Profiles Table

The guest_profiles table stores attributes specific to the guest role including preferred_price_range for budget filtering, preferred_property_types stored as JSON for flexible preference management,

travel_preferences stored as JSON for accommodation preferences, guest_verification_status tracking identity verification levels, and verification_date recording when guest verification was completed. This table maintains a one-to-one relationship with the users table through the user_id foreign key with unique constraint.

Guest profiles are created automatically when users make their first booking attempt, ensuring seamless user experience without requiring explicit profile creation. The JSON fields for preferences provide flexibility to store complex preference structures without rigid schema constraints, accommodating evolving business requirements for preference tracking.

Host Profiles Table

The host_profiles table stores host-specific business information including verification status, performance metrics, and payout details. Key attributes include host_verification_status tracking the host approval process, host_since recording when the user became a host, response_rate and response_time_hours tracking host responsiveness, acceptance_rate measuring booking acceptance, host_rating storing overall host performance rating, total_properties counting active listings, superhost_status indicating elite host designation, payout_method_id referencing preferred payout method, tax_id storing encrypted tax identification, and business_name and business_registration for commercial hosts.

Host profiles are created when users submit host applications and undergo verification, ensuring only approved users can list properties. The performance metrics enable the platform to track host quality and identify superhosts who provide exceptional service. The business fields accommodate professional property managers operating multiple listings.

Updated Foreign Key Relationships

The role-based architecture required updates to foreign key relationships throughout the schema. The properties table now references host_profiles.host_profile_id instead of users.user_id, ensuring only verified hosts can create listings. The bookings table references guest_profiles.guest_profile_id for role-specific attributes while maintaining user_id for audit trail and authentication. The payouts table references host_profiles.host_profile_id to ensure proper business logic for host disbursements.

This redesign increased the entity count from 25 primary entities to 30 tables (27 primary entities + 3 triple relationship junction tables) with the addition of guest_profiles and host_profiles, while maintaining the three triple relationships and one recursive relationship required by the assignment. The role-based architecture provides clear separation of business logic, supports multi-role scenarios essential to the Airbnb business model, enables efficient role-specific queries, and facilitates future expansion with additional roles.

Database Schema Implementation

The database implementation translates the conceptual design into 30 MySQL tables with comprehensive constraints and relationships. The schema follows Third Normal Form normalization principles to eliminate redundancy while maintaining query performance through strategic indexing and denormalization where appropriate.

Data Type Selection Strategy

Data type selection balances storage efficiency, data integrity, and query performance. Integer types use INT for primary and foreign keys with AUTO_INCREMENT for generated keys, providing a 32-bit range sufficient for expected growth. BIGINT is reserved for high-volume transactional tables if needed in future scaling. VARCHAR is sized appropriately per attribute: 255 for email and names, 200 for titles and addresses, 100 for shorter identifiers, and 50 for codes and categories.

TEXT type accommodates variable-length content without size constraints for descriptions, reviews, and messages.

DECIMAL types maintain precision for financial values using DECIMAL(10,2) for monetary amounts supporting values up to 99,999,999.99, DECIMAL(5,2) for percentages and rates from 0.00 to 100.00, and DECIMAL(3,2) for ratings from 0.00 to 5.00. The DATETIME type is used consistently for timestamps providing precision to seconds, with MySQL automatically handling timezone conversions. BOOLEAN type explicitly tracks yes/no flags for is_active, is_guest, is_host, and similar attributes. The JSON type provides flexible schema-less storage for preferences, settings, and variable attribute collections.

Table Creation Strategy

Tables were created in dependency order to satisfy foreign key constraints. The creation sequence begins with independent lookup tables (countries, property_types, property_amenities, review_categories, booking_status), followed by core entity tables (users, cities, addresses), then role-specific tables (guest_profiles, host_profiles, user_profiles), entity-dependent tables (properties, bookings, reviews, conversations), and finally junction tables for many-to-many and triple relationships.

The CREATE TABLE statements include comprehensive constraints defined inline with column definitions where possible. All tables use the InnoDB storage engine explicitly through ENGINE=InnoDB specification. Character set and collation are set at the database level to utf8mb4_unicode_ci for full Unicode support including emoji and international characters. Timestamps use DEFAULT CURRENT_TIMESTAMP for creation times and ON UPDATE CURRENT_TIMESTAMP for modification tracking where appropriate.

Key Table Implementations

The users table serves as the central entity connecting all platform activities. It implements role tracking through boolean flags while maintaining essential authentication and contact information. The table includes unique constraints on email and phone to prevent duplicate registrations, password_hash storing bcrypt-hashed passwords never in plain text, and comprehensive timestamps tracking account creation and modifications.

The properties table links hosts to listings with comprehensive property attributes. Key features include foreign keys to host_profiles, property_types, and addresses ensuring referential integrity, property characteristics including max_guests, bedrooms, bathrooms, and size_sqm, activation status through is_active boolean enabling soft deletion, and timestamps for creation and modification tracking.

The bookings table manages reservation transactions with complex business logic. Implementation details include foreign keys to guest_profiles, properties, and booking_status, date range fields for check_in_date and check_out_date with validation, number_of_guests and total_amount capturing booking details, and confirmation timestamp tracking booking approval.

The property_pricing table implements dynamic pricing with date ranges. Features include foreign key to properties for linking pricing rules, start_date and end_date defining pricing period validity, base_price_per_night and optional weekend_price_per_night, weekly_discount_percentage and monthly_discount_percentage for long stays, minimum_stay_nights and maximum_stay_nights enforcing booking rules, and is_available flag controlling whether dates can be booked.

Constraint Implementation

Comprehensive constraints enforce business rules and maintain data integrity throughout the database. The constraint strategy encompasses primary keys for entity identification, foreign keys

for referential integrity, check constraints for value validation, unique constraints preventing duplicates, and not null constraints ensuring required data.

Primary Key Constraints

Every table implements a single-column surrogate primary key using INT AUTO_INCREMENT. This approach provides several advantages: simplified foreign key relationships using single columns, guaranteed uniqueness without composite key complexity, improved indexing performance with integer keys, and consistent key structure across the entire schema. Primary keys are named descriptively following the pattern tablename_id for clarity and consistency.

Foreign Key Constraints

Foreign key constraints enforce referential integrity and define relationship cardinality. All foreign keys are implemented with explicit constraint names following the pattern FK_sourcetable_targettable for traceability. The CASCADE option is used selectively: ON DELETE RESTRICT is the default protecting against accidental deletions of referenced records, ON DELETE CASCADE is used for dependent entities that should be removed with their parent, and ON DELETE SET NULL is used where the relationship is optional.

Critical foreign key implementations include users to guest_profiles and host_profiles with CASCADE deletion since profiles are meaningless without users, properties to host_profiles with RESTRICT preventing deletion of hosts with active listings, bookings to properties and guest_profiles with RESTRICT protecting booking history, and reviews to bookings with RESTRICT maintaining review integrity.

Check Constraints

Check constraints validate data values at insertion and update. Key implementations include monetary value validation ensuring all prices and amounts are positive through constraints like CHECK (base_price_per_night > 0), rating value ranges restricting ratings between 1 and 5 through CHECK (rating >= 1 AND rating <= 5), percentage value ranges ensuring rates are between 0 and 100 through CHECK (response_rate >= 0 AND response_rate <= 100), and date logic validation ensuring end dates follow start dates through CHECK (end_date > start_date).

Additional business rule constraints include guest count validation ensuring number_of_guests does not exceed property max_guests, stay duration validation enforcing minimum and maximum night requirements, and status transition validation through ENUM types restricting status values to predefined lists.

Unique Constraints

Unique constraints prevent duplicate entries for business-critical fields. The users table enforces unique email addresses and phone numbers preventing multiple accounts with the same contact information. The user_profiles, guest_profiles, and host_profiles tables enforce unique user_id ensuring one-to-one relationships with the users table. Property photos enforce unique combinations of property_id and display_order preventing duplicate photo orderings. These constraints are implemented as indexes with UNIQUE keyword providing both constraint enforcement and query optimization.

Not Null Constraints

Not null constraints ensure data completeness for critical attributes. Every table includes NOT NULL on primary keys, foreign keys for mandatory relationships, and essential business attributes like names, dates, and amounts. Optional fields like middle names, secondary contact methods, and supplementary information allow NULL values, providing flexibility while maintaining data quality for core attributes.

Normalization Implementation

The database achieves Third Normal Form normalization, eliminating redundancy while maintaining practical query performance. The normalization process followed systematic analysis of functional dependencies, decomposition of non-normalized structures, and validation against normal form requirements.

First Normal Form Requirements

First Normal Form requires atomic values in all columns with no repeating groups. The schema satisfies 1NF through several design decisions. Atomic values are ensured by storing each discrete piece of information in separate columns. For example, names are split into `first_name` and `last_name` rather than a combined name field. Addresses are decomposed into `street_address`, `city`, `postal_code`, and coordinate components rather than a single address text field.

Repeating groups are eliminated through separate tables. Property amenities are stored in a junction table `property_amenity_links` rather than a repeating amenity field in properties. Property photos use a dedicated `property_photos` table with multiple rows per property rather than `photo1`, `photo2`, `photo3` columns. Multiple phone numbers or email addresses per user would require separate contact tables if needed.

Second Normal Form Requirements

Second Normal Form requires 1NF plus all non-key attributes must be fully dependent on the entire primary key. This particularly matters for tables with composite keys. The schema achieves 2NF through the use of surrogate primary keys in most tables, eliminating partial dependency issues. Junction tables like `property_amenity_links` use surrogate keys rather than composite keys of the two foreign keys, simplifying dependency analysis.

For tables where composite keys could theoretically exist, attributes are validated for full functional dependency. In `property_pricing`, all attributes depend on the complete combination of `property_id` and `date_range`, not just `property_id` alone. In `review_ratings`, the `rating_value` depends on both `review_id` and `category_id`, not either alone.

Third Normal Form Requirements

Third Normal Form requires 2NF plus no transitive dependencies where non-key attributes depend on other non-key attributes. The schema achieves 3NF through careful decomposition of transitive relationships. Location information demonstrates proper 3NF decomposition. Rather than storing `city_name` and `country_name` in the addresses table, addresses references the cities table which references the countries table, eliminating the transitive dependency `addresses → city → country`.

Similarly, property type information is normalized into the `property_types` table rather than storing type details in properties. Host performance metrics are stored in `host_profiles` rather than duplicated in properties. User contact preferences are separated into `user_profiles` rather than cluttering the users table. These decompositions eliminate transitive dependencies while maintaining clear relationships between entities.

Strategic Denormalization

While the schema targets 3NF, strategic denormalization is applied where performance benefits outweigh redundancy costs. The `host_profiles` table includes `total_properties` and `host_rating` as calculated fields to avoid expensive aggregation queries for every property listing display. The `bookings` table stores `total_amount` rather than calculating it from nightly rates and length of stay, capturing the actual amount charged at booking time even if pricing rules change later. These

denormalizations are carefully chosen to optimize critical query paths while maintaining data integrity through application logic or database triggers.

Functional Dependencies

Functional dependencies describe how attributes in a table are determined by other attributes. Understanding these dependencies is essential for normalization and maintaining data integrity. The schema's functional dependencies were systematically analyzed to ensure proper table decomposition and eliminate anomalies.

Primary Functional Dependencies

Every table exhibits a primary functional dependency where the primary key determines all other attributes. In the users table, `user_id` functionally determines `email`, `password_hash`, `first_name`, `last_name`, `phone`, `date_of_birth`, `role_flags`, and `timestamps`. This dependency is denoted as `user_id → (email, password_hash, first_name, last_name, phone, date_of_birth, is_guest, is_host, is_admin, created_at, updated_at, is_active)`.

For properties, `property_id` determines all property attributes: `property_id → (host_id, property_type_id, title, description, address_id, max_guests, bedrooms, bathrooms, size_sqm, created_at, updated_at, is_active)`. Similar primary dependencies exist for every entity table, forming the foundation of the relational schema.

Candidate Key Dependencies

Beyond primary keys, candidate keys provide alternative unique identifiers with their own functional dependencies. In users, `email` serves as a candidate key where `email → (user_id, all other user attributes)`. `Phone number` is another candidate key where `phone → (user_id, all other user attributes)`. These candidate keys are enforced through unique constraints and could theoretically serve as primary keys, though surrogate keys provide better performance.

Composite Dependencies in Junction Tables

Junction tables managing many-to-many relationships demonstrate composite functional dependencies. In `property_amenity_links`, the combination of `property_id` and `amenity_id` determines availability status and additional information: `(property_id, amenity_id) → (is_available, additional_info)`. Neither `property_id` nor `amenity_id` alone determines these attributes; only together do they form a unique identifier for a property-amenity association.

The `review_ratings` table shows similar composite dependency where `(review_id, category_id) → rating_value`. A review may have multiple category ratings, but for any review-category combination, there is exactly one rating value.

Transitive Dependencies Resolution

The normalization process eliminated transitive dependencies through table decomposition. Originally, addresses might have included `city_name` and `country_name`, creating a transitive dependency `address_id → city_id → city_name`, `country_id → country_name`. This was resolved by separating cities and countries into dedicated tables, with addresses referencing cities which reference countries, eliminating redundancy and update anomalies.

Integrity Rules and Business Logic

Database integrity rules enforce business requirements and maintain data consistency. The implementation combines declarative constraints in the schema with application-layer validation for complex multi-table rules that cannot be expressed in standard SQL constraints.

Entity Integrity

Entity integrity ensures each entity instance is uniquely identifiable and properly formed. Primary key constraints guarantee every record has a unique identifier that cannot be null. Unique constraints on candidate keys like email and phone prevent duplicate entities with different primary keys. Not null constraints on essential attributes ensure entities cannot exist in incomplete states.

Referential Integrity

Referential integrity maintains consistency across related tables. Foreign key constraints with appropriate ON DELETE and ON UPDATE actions enforce these rules automatically. Critical referential integrity rules include: properties must reference valid hosts through host_id FK to host_profiles; bookings must reference valid guests, properties, and statuses; reviews must reference valid users and bookings; payments must reference valid bookings and payment methods.

The constraint actions are carefully chosen based on business requirements. CASCADE deletion is used when child entities are meaningless without parents, such as deleting all user profiles when a user account is deleted. RESTRICT deletion is used to protect business-critical data like preventing property deletion if bookings exist. SET NULL is used sparingly where relationships can meaningfully become undefined.

Domain Integrity

Domain integrity ensures attribute values conform to valid ranges and formats. Data type selection provides the first level of domain integrity by restricting values to appropriate types. Check constraints enforce specific value ranges for monetary amounts that must be positive, ratings restricted to 1-5 scale, percentages limited to 0-100 range, and dates following logical ordering with end dates after start dates.

ENUM types provide domain integrity for status fields, restricting values to predefined sets like booking status values of pending, confirmed, cancelled, completed, or no_show. This prevents invalid status entries while providing clear documentation of allowed values.

Business Rule Enforcement

Complex business rules requiring multi-table validation are enforced through application logic rather than database constraints. Key business rules include: booking dates must not overlap for the same property, guests must not exceed property max_guests capacity, bookings must satisfy property minimum and maximum stay requirements, reviews can only be created after booking completion, users must be verified before listing properties, and payments must be processed before check-in dates.

These rules are implemented in the application layer with validation logic before database operations. The database schema supports these rules through appropriate attributes and relationships, while the application enforces them during business operations. This separation of concerns maintains database independence while enabling complex rule enforcement.

Test Data Population Strategy

Comprehensive test data was created to validate schema design, constraint enforcement, and query functionality. The test data strategy focused on creating realistic, interconnected data that demonstrates all system capabilities while testing edge cases and constraint validations.

Data Population Approach

Test data insertion followed the same dependency order as table creation, ensuring referential integrity constraints were satisfied. The population sequence began with lookup tables (countries,

cities, property types, amenities, review categories, booking statuses), followed by core entities (users, addresses), role-specific tables (guest and host profiles), dependent entities (properties with pricing and amenities, bookings, payments, payouts, reviews), and junction tables (triple relationships).

The test dataset includes 20 users with varied role combinations demonstrating multi-role support, 10 properties across different types and locations showing geographic diversity, 15 bookings in various states (pending, confirmed, completed, cancelled) covering the full booking lifecycle, and comprehensive relationships including pricing rules, amenities, photos, reviews, messages, and notifications demonstrating all system features.

Multi-Role User Scenarios

The test data demonstrates the recursive relationship where users can simultaneously act as both guests and hosts. User 1 (John Doe) is both a guest and host, with an active host profile managing properties while also booking other properties as a guest. User 3 (Mike Johnson) exhibits the same dual-role behavior. Several users are guest-only, representing travelers who do not list properties. Other users are host-only, representing property managers focused on hosting without booking other properties.

This variety validates the role-based architecture and demonstrates that the schema properly handles all role combinations without conflicts or anomalies.

Triple Relationship Demonstrations

The test data includes complete examples of all three required triple relationships. The `property_booking_pricing` triple relationship connects properties, bookings, and pricing rules, showing which pricing configuration was applied to each booking. The `user_booking_review` triple relationship demonstrates bidirectional reviews where both guests and hosts review each other after booking completion. The `booking_payment_payout` triple relationship tracks the complete financial chain from guest payment through platform commission to host payout.

Each triple relationship includes multiple examples with different scenarios, validating that the junction tables properly capture the complex many-to-many-to-many relationships required by the business model.

Constraint Validation Testing

Test data insertion specifically tested constraint enforcement. Attempts to insert invalid data verified that: check constraints properly reject negative prices and out-of-range ratings, foreign key constraints prevent references to non-existent entities, unique constraints block duplicate emails and phone numbers, and not null constraints require essential fields. These validation tests confirmed that the constraint implementation properly enforces data integrity rules.

Query Implementation and Testing

Comprehensive SQL queries were developed to demonstrate database functionality and validate the schema design. The query suite includes simple single-table retrievals, complex multi-table joins, aggregate analysis, and demonstrations of all three triple relationships and the recursive relationship.

Basic Retrieval Queries

Simple `SELECT` queries retrieve data from individual tables to verify basic functionality. These queries demonstrate successful data population and proper column naming. Examples include retrieving active users with role flags, displaying guest profiles with verification status, showing host

profiles with performance metrics, listing properties with titles and capacity details, and viewing bookings with dates and amounts.

These foundational queries validate that the schema structure is correct, data types are appropriate, and constraints allow proper data insertion. They serve as building blocks for more complex queries.

Relationship Join Queries

Multi-table join queries demonstrate relationships and retrieve connected data across entities. Key relationship queries include properties joined with hosts, addresses, cities, and countries showing complete property location information; bookings joined with guests, properties, and hosts displaying full reservation details; reviews joined with reviewers, reviewees, and bookings showing complete review context; and conversations joined with participants showing message exchange information.

These queries validate foreign key relationships and demonstrate that the normalized schema can efficiently reconstruct related information. The query results confirmed that relationships are properly established and join operations return expected data.

Triple Relationship Queries

Specialized queries demonstrate the three required triple relationships. The `property_booking_pricing` query joins properties, bookings, and `property_pricing` through the `property_booking_pricing` junction table, showing which pricing rule was applied to each booking with the actual price charged. The `user_booking_review` query demonstrates bidirectional reviews by joining users, bookings, and reviews through the `user_booking_review` junction table, displaying review role (reviewer or reviewee) and interaction type. The `booking_payment_payout` query traces the complete financial flow by joining bookings, payments, and payouts through the `booking_payment_payout` junction table, showing transaction chain status from guest payment to host payout.

These queries validate the complex many-to-many-to-many relationships and demonstrate that the junction table approach successfully models triple relationships while maintaining data integrity and query efficiency.

Recursive Relationship Query

The recursive relationship query identifies users who hold both guest and host roles simultaneously. The query selects from users where both `is_guest` and `is_host` flags are TRUE, demonstrating the self-referential many-to-many relationship where users can book properties from other users while also hosting their own properties. The results show multiple users exhibiting this dual-role behavior, validating that the schema properly supports the recursive relationship required by the assignment.

Aggregate and Analytical Queries

Analytical queries demonstrate the schema's capability to support business intelligence needs. Examples include counting total tables in the database schema, aggregating booking totals by property, calculating average ratings by host, summarizing payment totals by month, and analyzing occupancy rates by property type. These queries validate that the schema supports not only transactional operations but also analytical workloads essential for business reporting and decision-making.

Implementation Challenges and Solutions

The implementation process encountered several challenges that required careful problem-solving and design refinements. These challenges and their solutions provide valuable lessons for future database projects.

Role-Based Architecture Complexity

Challenge: The initial design lacked clear separation between guest and host roles, making it difficult to implement role-specific business logic and track role-specific attributes. The tutor feedback identified this critical deficiency requiring significant schema redesign.

Solution: The role-based architecture was implemented through dedicated `guest_profiles` and `host_profiles` tables with one-to-one relationships to users, role tracking flags in the users table to indicate active roles, and updated foreign key relationships throughout the schema to reference appropriate profile tables. This solution provides clear separation of concerns while supporting multi-role users essential to the platform business model.

Foreign Key Dependency Management

Challenge: Creating tables and inserting data in the correct order to satisfy foreign key constraints proved complex with 30 interconnected tables. Circular dependencies between tables could have prevented successful database creation.

Solution: A dependency graph was created to visualize table relationships and identify the correct creation order. Tables were created in a carefully planned sequence starting with independent lookup tables, progressing through core entities, and ending with junction tables. This systematic approach ensured foreign keys could be properly established without errors.

Triple Relationship Implementation

Challenge: Modeling three-way many-to-many-to-many relationships through junction tables while maintaining clarity and query efficiency required careful design. The junction tables needed to properly capture the semantics of each triple relationship.

Solution: Each triple relationship was implemented with a dedicated junction table containing foreign keys to all three participating entities plus additional attributes specific to that relationship. The `property_booking_pricing` table includes `applied_price` and `pricing_rule_applied` attributes. The `user_booking_review` table includes `review_role` and `interaction_type` attributes. The `booking_payment_payout` table includes `transaction_chain_status` attribute. This approach makes the relationships explicit and queryable while maintaining normalization.

Performance vs Normalization Trade-offs

Challenge: Strict adherence to Third Normal Form can create performance issues for frequently executed queries requiring multiple joins. The balance between normalization and query efficiency needed careful consideration.

Solution: Strategic denormalization was applied in specific cases where performance benefits justified redundancy. The `host_profiles` table includes `total_properties` and `host_rating` as cached calculations avoiding expensive aggregations. The `bookings` table stores `total_amount` capturing the actual price at booking time rather than recalculating from nightly rates. These denormalizations are carefully chosen and documented, with application logic or triggers maintaining consistency between redundant data.

JSON vs Relational Data

Challenge: Certain attributes like user preferences and property amenity details have variable structure that could be modeled either as JSON fields or through additional normalized tables. The choice impacts schema flexibility, query complexity, and data integrity.

Solution: A hybrid approach was adopted using JSON fields for truly variable structures where the exact attributes cannot be predetermined (user preferences, notification settings) while maintaining relational tables for structured many-to-many relationships (property amenities through junction table). This provides flexibility for variable data while maintaining referential integrity and query efficiency for structured relationships.

Phase 2 Tutor Feedback Response

The Phase 2 submission received positive feedback with specific requirements for the final phase submission. The tutor feedback identified components that should be included in the final deliverable to complete the comprehensive database portfolio.

Missing Documentation Components

Feedback: The Phase 2 submission lacked an updated data dictionary and updated Entity-Relationship Diagram reflecting the refined schema based on Phase 1 feedback. These documentation components are essential for the final portfolio submission.

Response: The updated data dictionary and ERD will be included in the final phase submission. The data dictionary will comprehensively document all 30 tables (27 primary entities + 3 triple relationship junction tables) with their attributes, data types, constraints, and relationships. The updated ERD will reflect the role-based architecture with guest_profiles and host_profiles entities, showing all relationships including the three triple relationships and recursive relationship. These documents will be professionally formatted and properly integrated into the final portfolio documentation.

Enhanced Test Case Requirements

Feedback: The Phase 2 test queries primarily demonstrated simple SELECT statements from individual tables rather than comprehensive test cases. True test cases should extract data from more than three tables using JOIN operations to demonstrate complex query capabilities and validate relationship implementations.

Response: Enhanced test cases will be developed for the final submission demonstrating complex multi-table joins. These test cases will include queries joining four or more tables to retrieve complete business scenarios, queries demonstrating all three triple relationships with multiple joins, queries showing the recursive relationship through self-referencing logic, and analytical queries combining joins with aggregations for business intelligence. The test cases will be documented with query explanations, execution results, and analysis of what each test demonstrates about the database design.

Final Submission Checklist

Based on the tutor feedback, the final phase submission will include all Phase 2 components (SQL schema with all 27 tables, comprehensive constraints and relationships, test data population scripts, and Phase 2 documentation), plus the updated comprehensive data dictionary documenting all entities and attributes, updated Entity-Relationship Diagram showing the refined schema, enhanced test cases with complex multi-table JOIN queries, complete integration of Phase 1, Phase 2, and Phase 3 documentation, and professional formatting with consistent style throughout all phases.

This response ensures the final submission addresses all feedback points while maintaining the high-quality implementation completed in Phase 2. The additions enhance documentation completeness without requiring changes to the database design itself, which received positive feedback from the tutor.

Phase 2 Summary and Outcomes

Phase 2 successfully transformed the conceptual Entity-Relationship model from Phase 1 into a fully functional MySQL database. The implementation incorporated all tutor feedback, resulting in a robust, normalized schema that meets all assignment requirements while exceeding minimum expectations.

Key Accomplishments

The implementation delivered a comprehensive database schema with 27 normalized entities achieving Third Normal Form, comprehensive constraints including primary keys, foreign keys, check constraints, unique constraints, and not null constraints, role-based user architecture with `guest_profiles` and `host_profiles` addressing Phase 1 feedback, three triple relationships (property-booking-pricing, user-booking-review, booking-payment-payout) properly implemented through junction tables, one recursive relationship demonstrating multi-role user support, realistic test data demonstrating all features and relationships, and comprehensive query suite validating functionality and performance.

Technical Achievements

The implementation demonstrated strong technical skills in database design and SQL. Data type selection optimized storage efficiency while maintaining data integrity. Constraint implementation enforced business rules through declarative schema definitions. Normalization eliminated redundancy while maintaining query performance. Foreign key relationships established clear entity connections with appropriate cascade behaviors. The schema supports both transactional and analytical workloads through proper indexing and design.

Business Value

The database provides a solid foundation for a production Airbnb-like platform. The role-based architecture supports real-world scenarios where users simultaneously act as guests and hosts. The triple relationships properly model complex business processes like dynamic pricing, bidirectional reviews, and financial transaction chains. The normalized design ensures data integrity while supporting business growth and feature expansion. Comprehensive constraints protect data quality and enforce business rules automatically.

Readiness for Phase 3

The Phase 2 implementation provides a complete database ready for Phase 3 enhancements. The schema is documented, tested, and validated through comprehensive queries. All relationships are properly established and demonstrated. Constraints ensure data integrity for production use. The design supports future expansion with additional features and scaling requirements. Phase 3 can proceed with confidence that the technical foundation is solid and production-ready.

Lessons Learned

The Phase 2 experience reinforced several important database design principles. Role-based architecture requires careful planning to separate concerns while supporting multi-role scenarios. Normalization must be balanced with performance through strategic denormalization. Foreign key dependencies require systematic planning to avoid circular references. Triple relationships need dedicated junction tables with clear semantics. Comprehensive constraints enforce data integrity but require thoughtful design. Test data should cover all scenarios including edge cases. Documentation must be detailed and professional to communicate design decisions effectively.

Phase 2 established a robust technical foundation for the Airbnb database system. The implementation successfully addressed all Phase 1 feedback while meeting assignment requirements for entities, relationships, normalization, and constraints. The database is ready for integration into a complete application platform, demonstrating professional-level database design and implementation skills.

Phase 2: Database Implementation

DLBDSPBDM01 - Build a Data Mart in SQL

SQL Query Documentation

Nikolas Daniel Vincenti

Implementation Summary

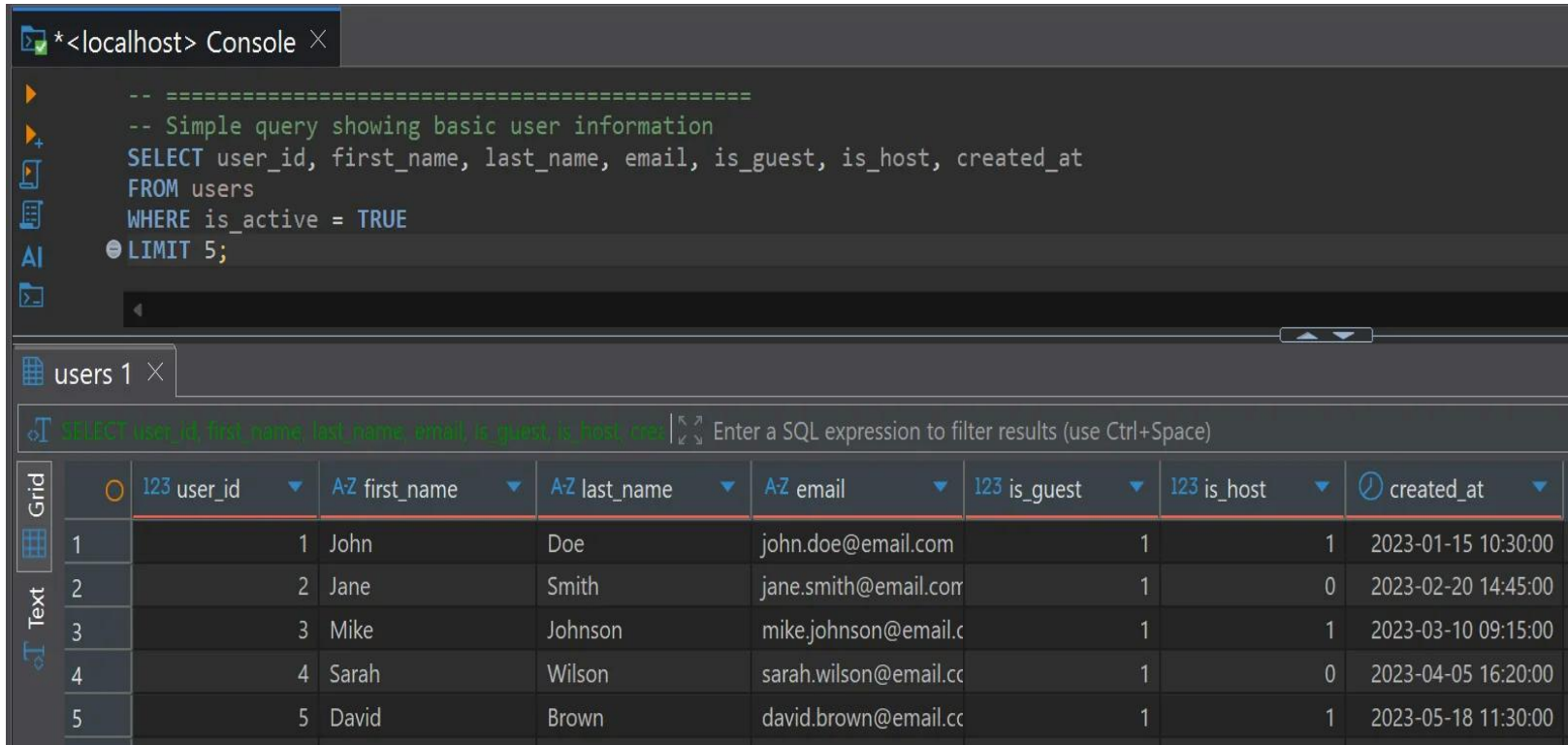
This presentation documents the SQL implementation of the database design from Phase 1 (revised on the basis of the received feedback).

Contents:

- 32 SQL query implementations
- Table creation and relationships
- Data insertion and manipulation
- Query results and validations

Basic User Table

Retrieves basic user information for active users showing names, email, and role flags



The screenshot displays a database interface with two main sections. The top section is a console window titled '*<localhost> Console' containing a SQL query. The bottom section is a results viewer titled 'users 1' showing the output of the query in a table format.

SQL Query:

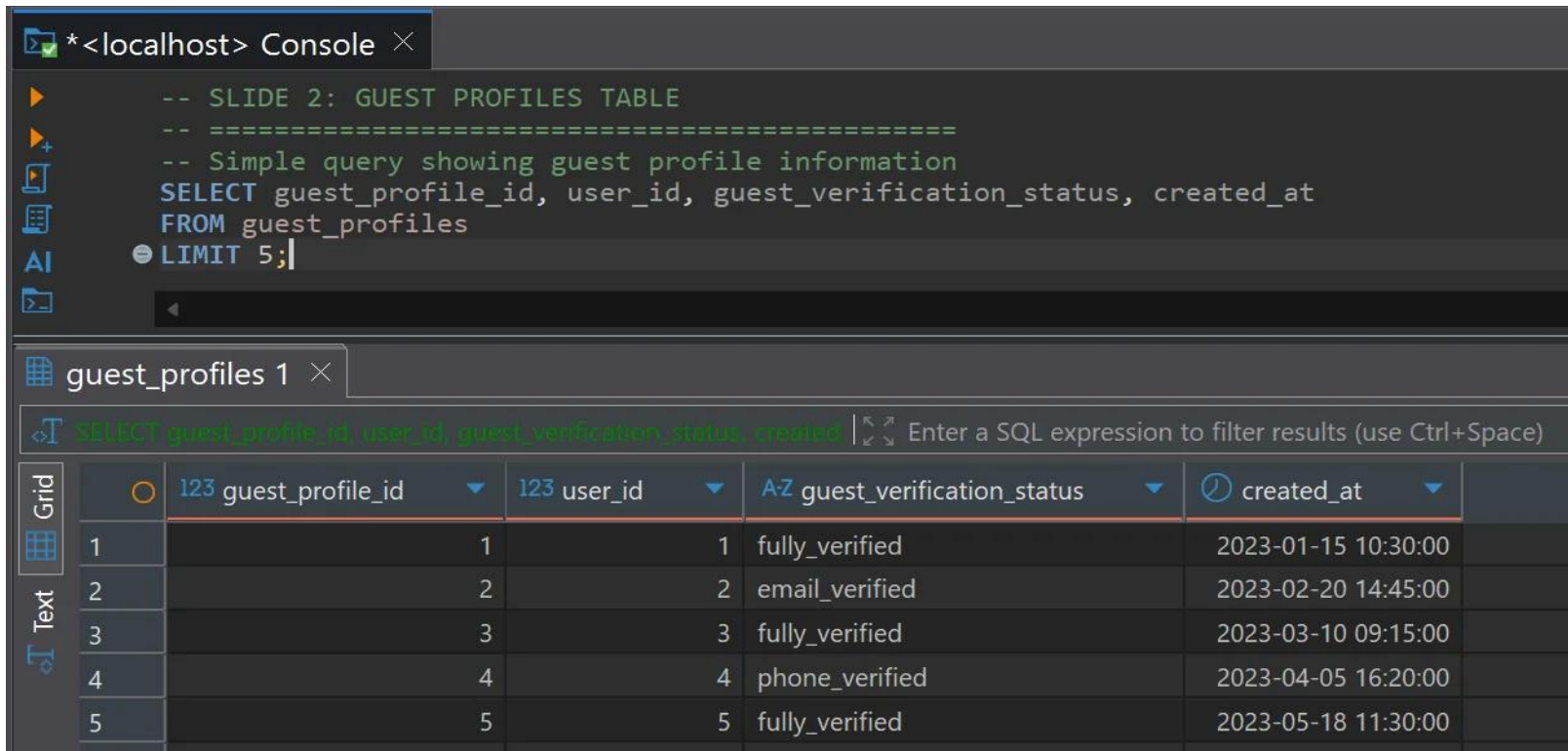
```
-- =====  
-- Simple query showing basic user information  
SELECT user_id, first_name, last_name, email, is_guest, is_host, created_at  
FROM users  
WHERE is_active = TRUE  
LIMIT 5;
```

Query Results:

	123 user_id	AZ first_name	AZ last_name	AZ email	123 is_guest	123 is_host	🕒 created_at
1	1	John	Doe	john.doe@email.com	1	1	2023-01-15 10:30:00
2	2	Jane	Smith	jane.smith@email.com	1	0	2023-02-20 14:45:00
3	3	Mike	Johnson	mike.johnson@email.com	1	1	2023-03-10 09:15:00
4	4	Sarah	Wilson	sarah.wilson@email.com	1	0	2023-04-05 16:20:00
5	5	David	Brown	david.brown@email.com	1	1	2023-05-18 11:30:00

Guest Profiles Table

Displays guest profile data including verification status and creation dates

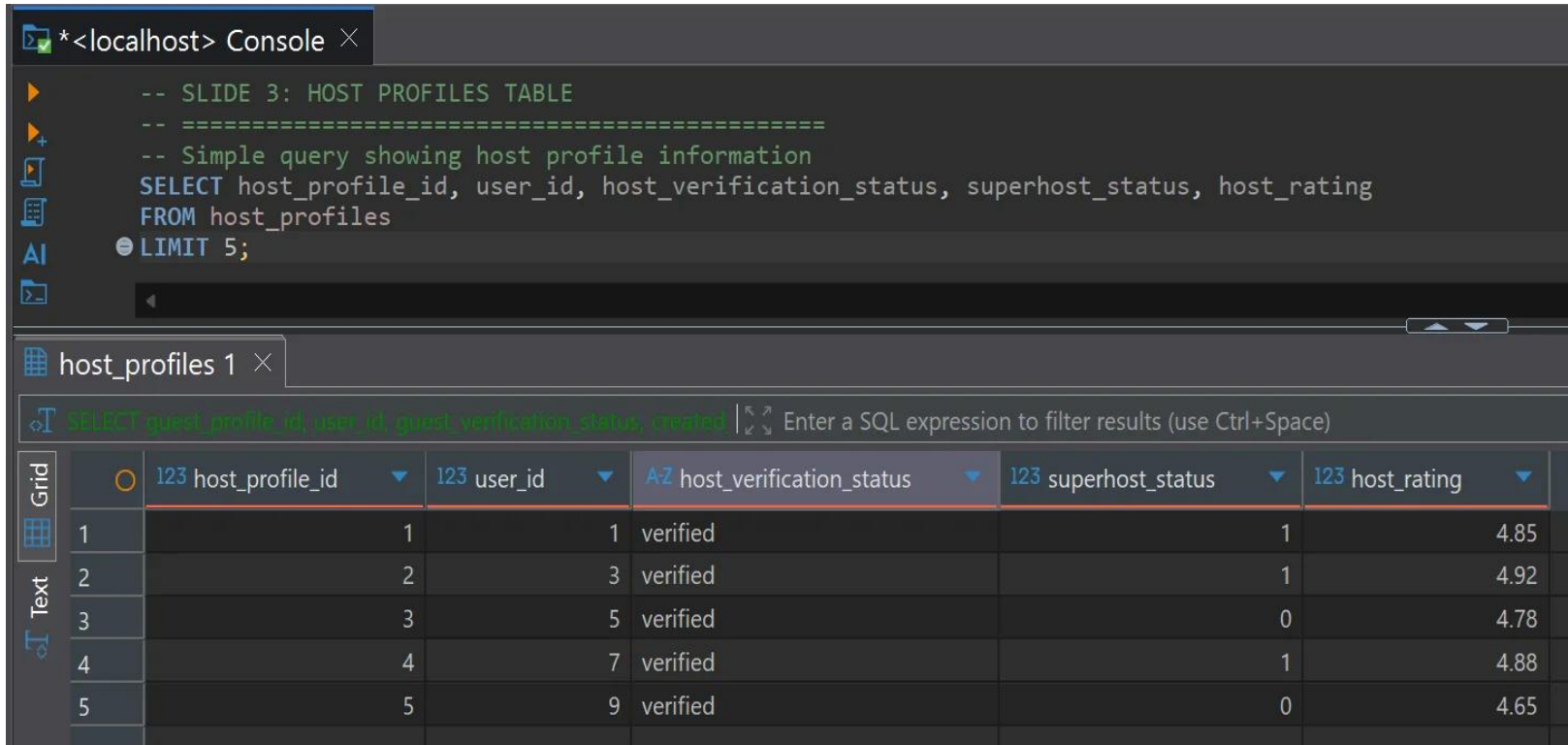


The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 2: GUEST PROFILES TABLE`, `-- =====`, `-- Simple query showing guest profile information`, `SELECT guest_profile_id, user_id, guest_verification_status, created_at`, `FROM guest_profiles`, and `LIMIT 5;`. The bottom pane, titled 'guest_profiles 1', displays the results of the query in a table. The table has columns for 'guest_profile_id', 'user_id', 'guest_verification_status', and 'created_at'. The results are shown in a grid view with 5 rows of data.

	123 guest_profile_id	123 user_id	A-Z guest_verification_status	created_at
1	1	1	fully_verified	2023-01-15 10:30:00
2	2	2	email_verified	2023-02-20 14:45:00
3	3	3	fully_verified	2023-03-10 09:15:00
4	4	4	phone_verified	2023-04-05 16:20:00
5	5	5	fully_verified	2023-05-18 11:30:00

Host Profiles Table

Shows host profile information with verification status, superhost status, and ratings

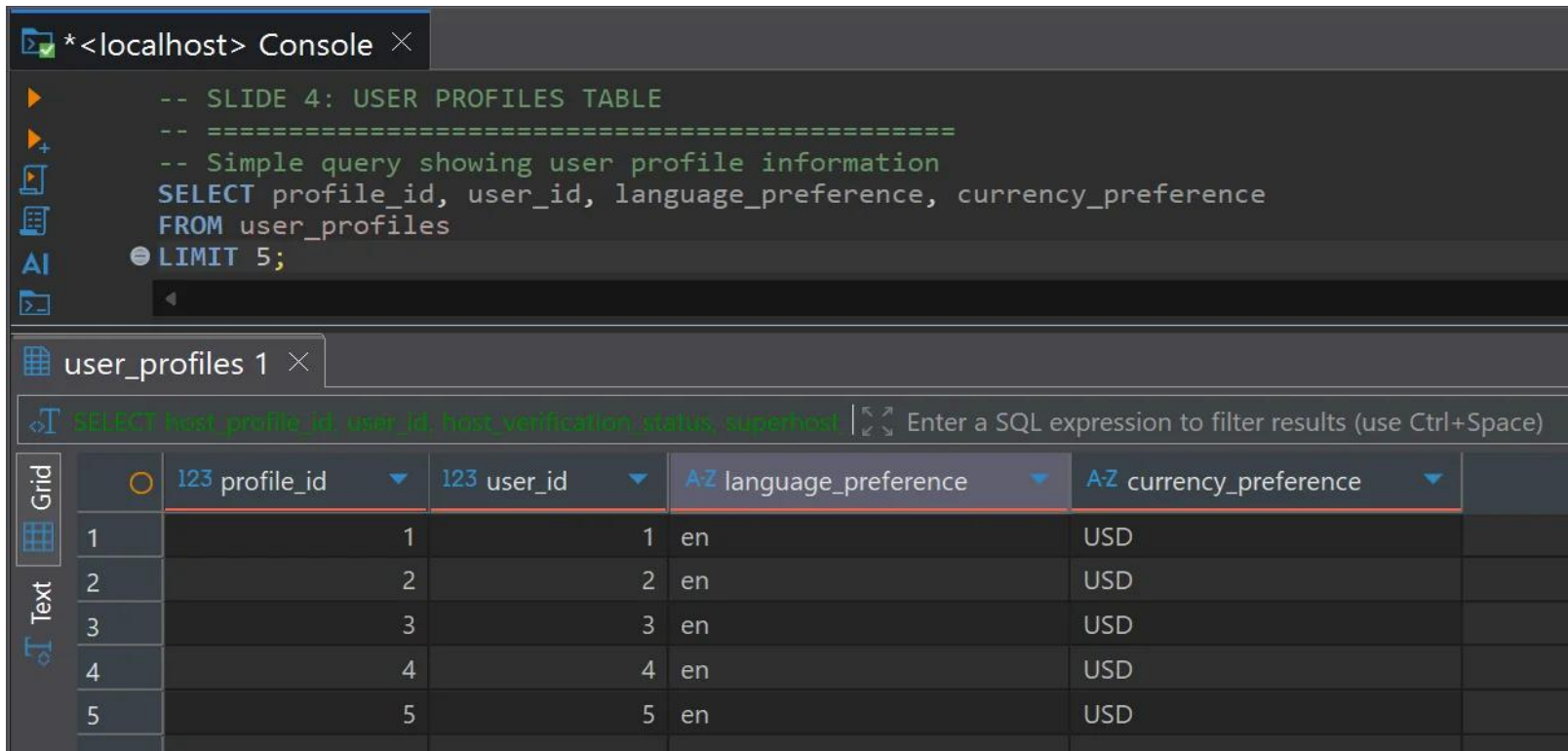


The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 3: HOST PROFILES TABLE`, `-- =====`, `-- Simple query showing host profile information`, `SELECT host_profile_id, user_id, host_verification_status, superhost_status, host_rating`, `FROM host_profiles`, and `LIMIT 5;`. The bottom pane, titled 'host_profiles 1', displays the results of the query in a table. The table has columns: host_profile_id, user_id, host_verification_status, superhost_status, and host_rating. The results show 5 rows of data.

	123 host_profile_id	123 user_id	A-Z host_verification_status	123 superhost_status	123 host_rating
1	1	1	verified	1	4.85
2	2	3	verified	1	4.92
3	3	5	verified	0	4.78
4	4	7	verified	1	4.88
5	5	9	verified	0	4.65

User Profiles Table

Lists user preferences for language and currency settings



The screenshot displays a database interface with two main sections. The top section is a console window titled '*<localhost> Console' containing a SQL query. The bottom section is a table viewer titled 'user_profiles 1' showing the results of the query in a grid format.

Console Window:

```
-- SLIDE 4: USER PROFILES TABLE
-- =====
-- Simple query showing user profile information
SELECT profile_id, user_id, language_preference, currency_preference
FROM user_profiles
LIMIT 5;
```

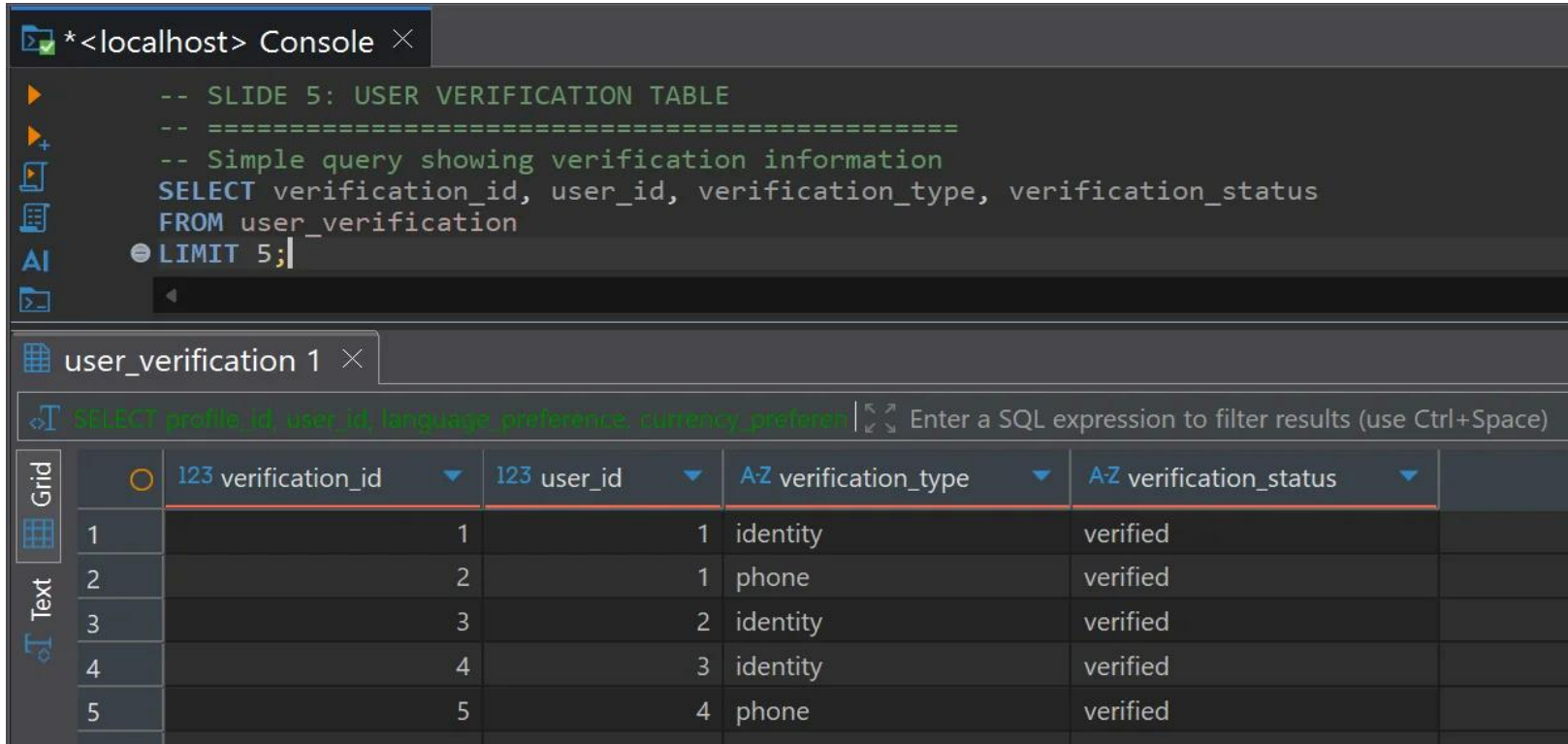
Table Viewer:

SQL Expression: `SELECT last_profile_id, user_id, last_verification_status, superhost` | Enter a SQL expression to filter results (use Ctrl+Space)

	123 profile_id	123 user_id	A-Z language_preference	A-Z currency_preference
1	1	1	en	USD
2	2	2	en	USD
3	3	3	en	USD
4	4	4	en	USD
5	5	5	en	USD

User Verification Table

Displays user verification records showing verification types and status

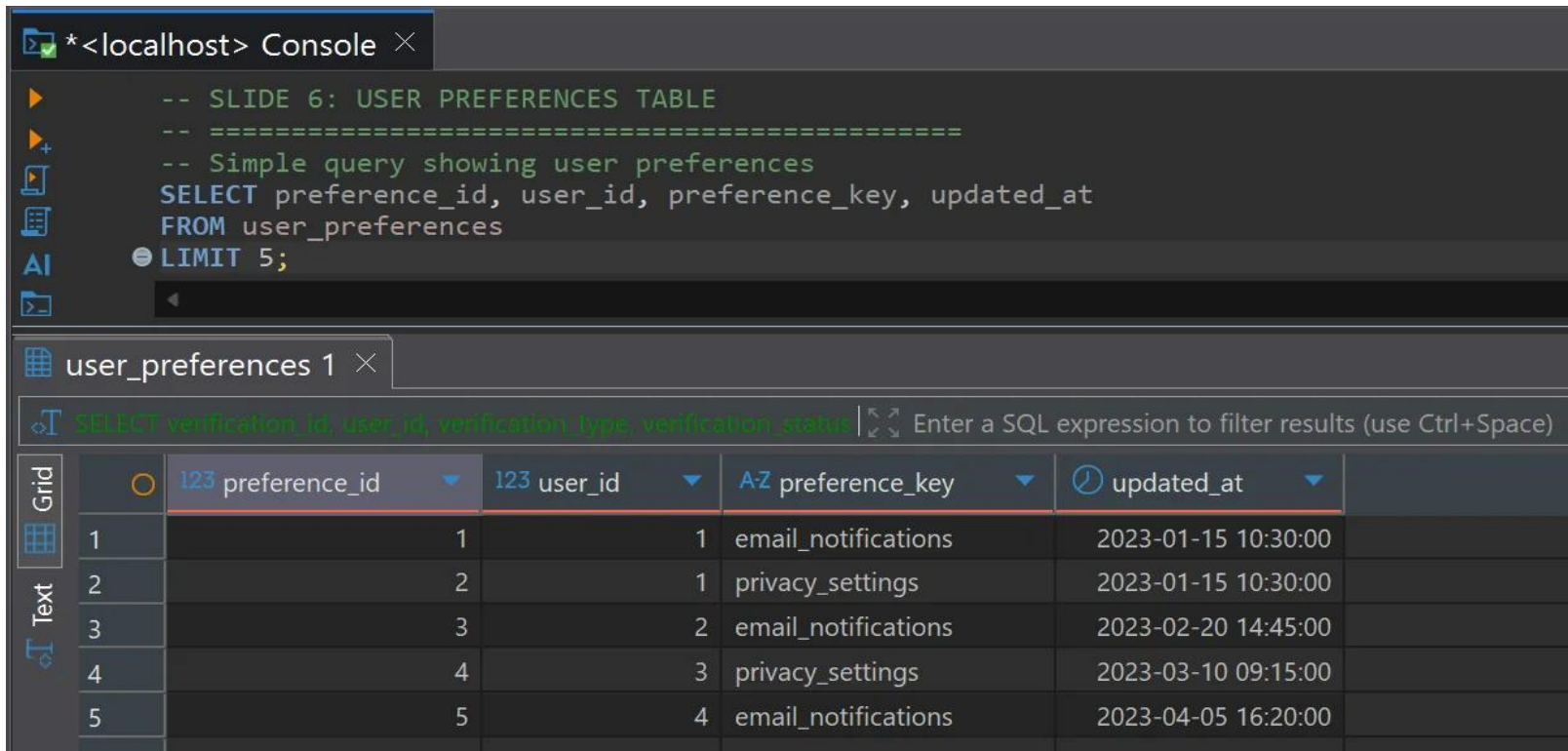


The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 5: USER VERIFICATION TABLE`, `-- =====`, `-- Simple query showing verification information`, `SELECT verification_id, user_id, verification_type, verification_status`, `FROM user_verification`, and `LIMIT 5;`. The bottom pane, titled 'user_verification 1', shows the results of the query in a table view. The table has columns for 'verification_id', 'user_id', 'verification_type', and 'verification_status'. The results are as follows:

	123 verification_id	123 user_id	A-Z verification_type	A-Z verification_status
1	1	1	identity	verified
2	2	1	phone	verified
3	3	2	identity	verified
4	4	3	identity	verified
5	5	4	phone	verified

User Preferences Table

Shows user preference settings with keys and last update timestamps



The screenshot shows a database console window with a query and its results. The console window has a tab labeled '*<localhost> Console'. The query is as follows:

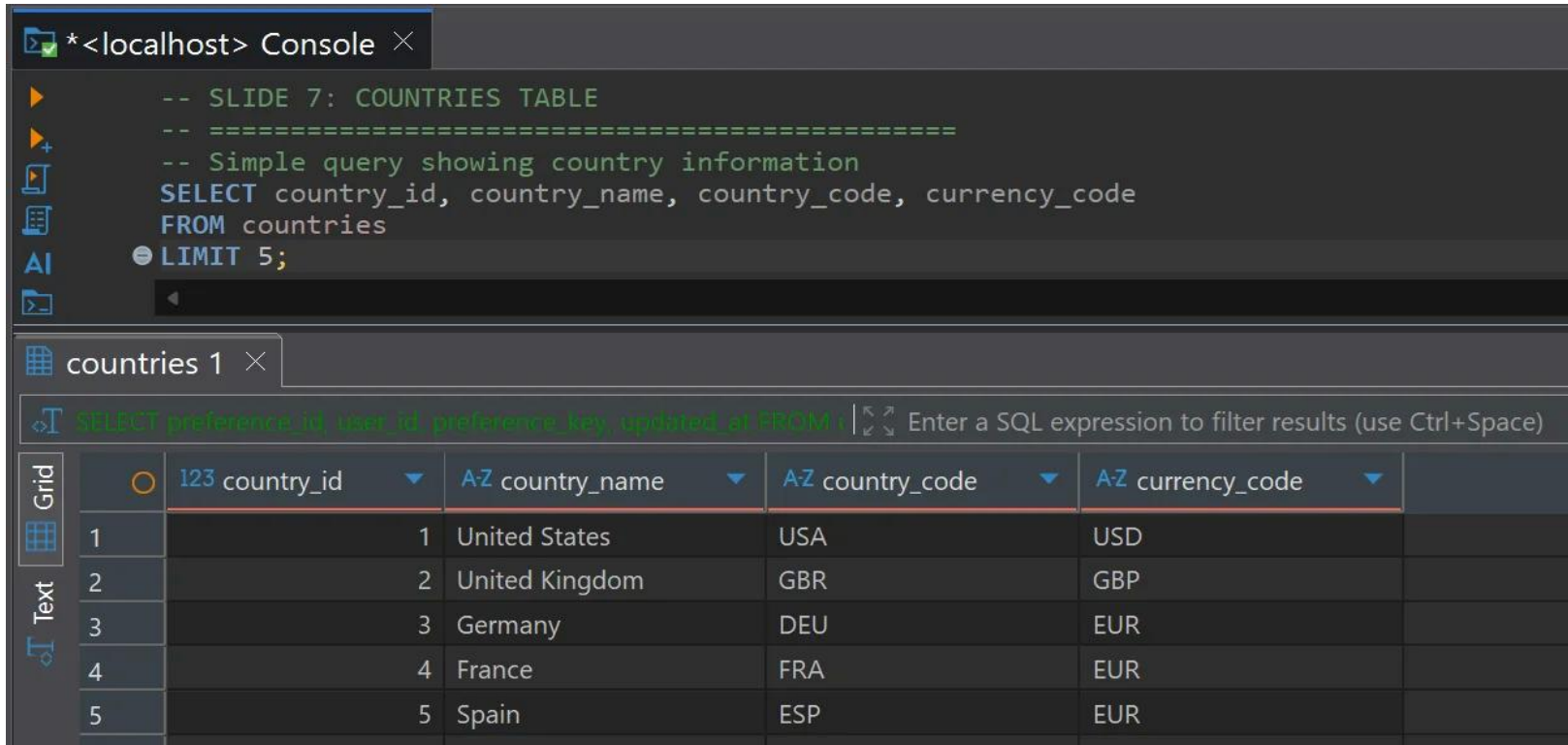
```
-- SLIDE 6: USER PREFERENCES TABLE
-- =====
-- Simple query showing user preferences
SELECT preference_id, user_id, preference_key, updated_at
FROM user_preferences
LIMIT 5;
```

Below the console, there is a results window titled 'user_preferences 1'. It shows a table with 5 rows. The table has columns: preference_id, user_id, preference_key, and updated_at. The first row is highlighted. The table is displayed in 'Grid' view, with a 'Text' view option also visible.

	123 preference_id	123 user_id	A-Z preference_key	updated_at
1	1	1	email_notifications	2023-01-15 10:30:00
2	2	1	privacy_settings	2023-01-15 10:30:00
3	3	2	email_notifications	2023-02-20 14:45:00
4	4	3	privacy_settings	2023-03-10 09:15:00
5	5	4	email_notifications	2023-04-05 16:20:00

Countries Table

Lists country information with names, codes, and currency codes



The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query. The bottom pane, titled 'countries 1', displays the results of the query in a grid view.

SQL Query:

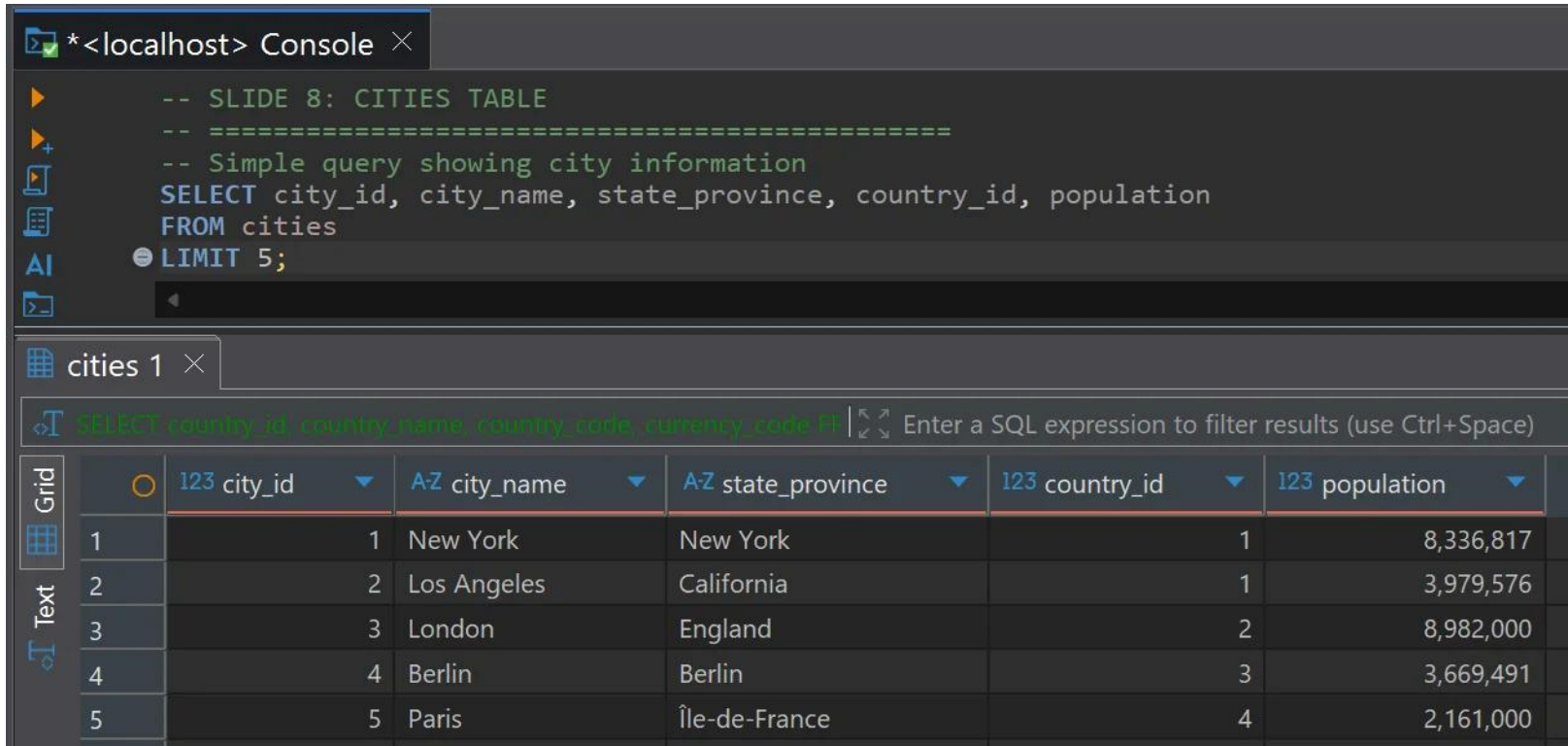
```
-- SLIDE 7: COUNTRIES TABLE
-- =====
-- Simple query showing country information
SELECT country_id, country_name, country_code, currency_code
FROM countries
LIMIT 5;
```

Query Results:

	123 country_id	A-Z country_name	A-Z country_code	A-Z currency_code
1	1	United States	USA	USD
2	2	United Kingdom	GBR	GBP
3	3	Germany	DEU	EUR
4	4	France	FRA	EUR
5	5	Spain	ESP	EUR

Cities Table

Displays city data including names, states/provinces, countries, and population

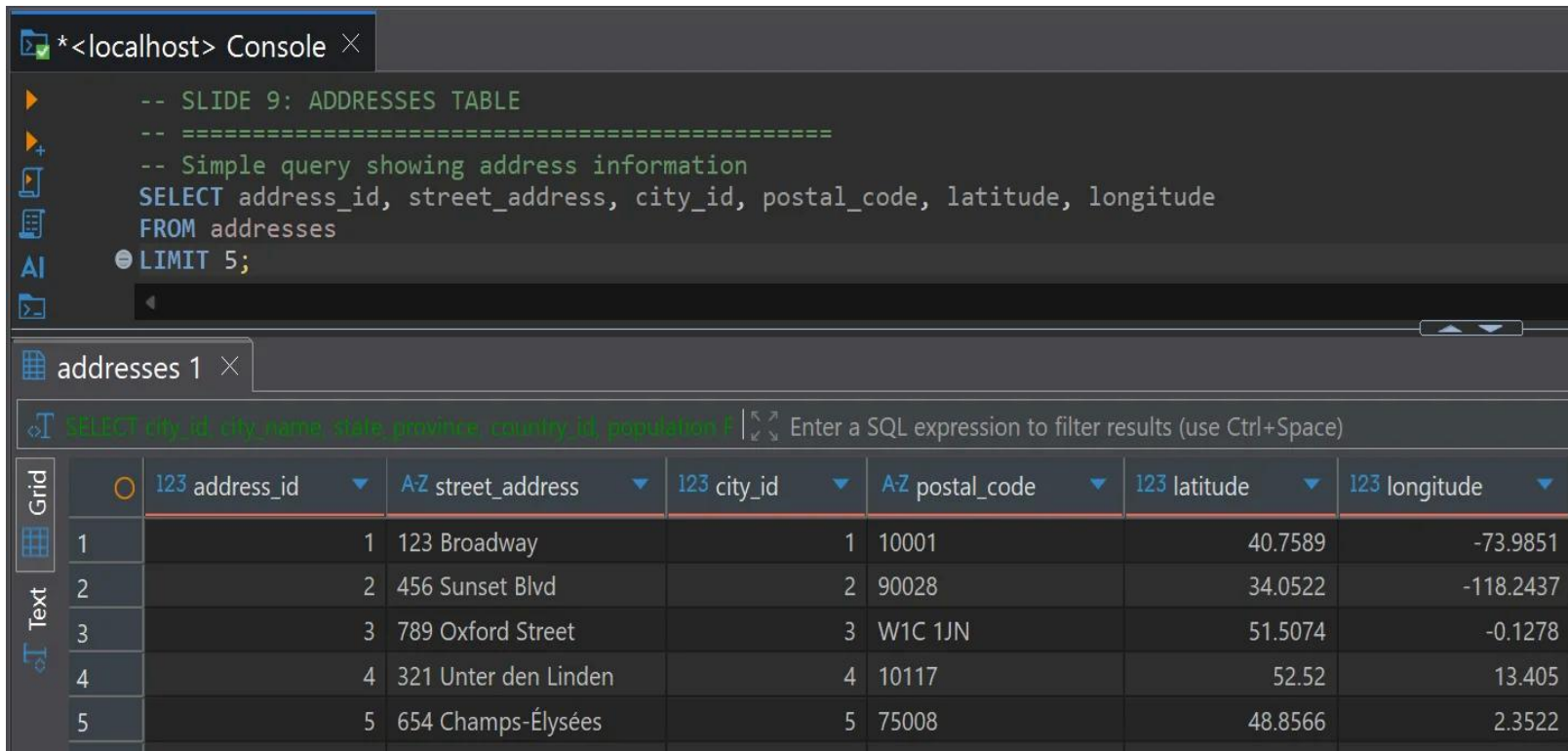


The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 8: CITIES TABLE`, `-- =====`, `-- Simple query showing city information`, `SELECT city_id, city_name, state_province, country_id, population`, `FROM cities`, and `LIMIT 5;`. The bottom pane, titled 'cities 1', shows the results of the query in a table. The table has 7 columns: an index, `city_id`, `city_name`, `state_province`, `country_id`, and `population`. The results are displayed in a grid view with 5 rows of data.

	123 city_id	A-Z city_name	A-Z state_province	123 country_id	123 population
1	1	New York	New York	1	8,336,817
2	2	Los Angeles	California	1	3,979,576
3	3	London	England	2	8,982,000
4	4	Berlin	Berlin	3	3,669,491
5	5	Paris	Île-de-France	4	2,161,000

Addresses Table

Shows address records with street addresses, postal codes, and coordinates



The screenshot displays a database interface with two main panels. The top panel, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 9: ADDRESSES TABLE`, `-- =====`, `-- Simple query showing address information`, `SELECT address_id, street_address, city_id, postal_code, latitude, longitude`, `FROM addresses`, and `LIMIT 5;`. The bottom panel, titled 'addresses 1', shows the results of the query in a table format. The table has 8 columns: 'address_id', 'street_address', 'city_id', 'postal_code', 'latitude', and 'longitude'. The results are displayed in a grid view with 5 rows of data.

	address_id	street_address	city_id	postal_code	latitude	longitude
1	1	123 Broadway	1	10001	40.7589	-73.9851
2	2	456 Sunset Blvd	2	90028	34.0522	-118.2437
3	3	789 Oxford Street	3	W1C 1JN	51.5074	-0.1278
4	4	321 Unter den Linden	4	10117	52.52	13.405
5	5	654 Champs-Élysées	5	75008	48.8566	2.3522

Property Types Table

Lists available property types with names and descriptions

The screenshot shows a database application interface. At the top, a console window titled '*<localhost> Console' contains a SQL query: `-- SLIDE 10: PROPERTY TYPES TABLE`, `-- =====`, `-- Simple query showing property types`, `SELECT property_type_id, type_name, description`, `FROM property_types`, and `LIMIT 5;`. Below the console, a window titled 'property_types 1' displays a table. The table has columns: 'property_type_id', 'type_name', and 'description'. The data is as follows:

	property_type_id	type_name	description
1	1	Apartment	Self-contained residential unit in a building
2	2	House	Detached residential building
3	3	Villa	Luxury detached house with garden
4	4	Condo	Individually owned unit in a building
5	5	Studio	Single room with kitchen and bathroom

Property Amenities Table

Displays property amenities with categories and descriptions

*<localhost> Console

```
-- SLIDE 11: PROPERTY AMENITIES TABLE
-- =====
-- Simple query showing property amenities
SELECT amenity_id, amenity_name, amenity_category, description
FROM property_amenities
LIMIT 5;
```

property_amenities 1

SELECT property

Connection: localhost
Time: 2025-10-14 14:33:38.928
Query: -- SLIDE 11: PROPERTY AMENITIES TABLE
-- =====
-- Simple query showing property amenities
SELECT amenity_id, amenity_name, amenity_category, description
FROM property_amenities
LIMIT 5

Enter a SQL expression to filter results (use Ctrl+Space)

	123 amenity_id	amenity_name	amenity_category	A-Z description
1	1	WiFi	Internet	High-speed internet connection
2	2	Kitchen	Kitchen	Fully equipped kitchen
3	3	Parking	Parking	Free parking on premises
4	4	Pool	Pool	Private or shared swimming pool
5	5	Gym	Gym	Fitness center or gym equipment

Properties Table

Shows active properties with host information, titles, and room details

*<localhost> Console X

AI

```
-- SLIDE 12: PROPERTIES TABLE
-- =====
-- Simple query showing properties
SELECT property_id, host_id, title, max_guests, bedrooms, bathrooms, is_active
FROM properties
WHERE is_active = TRUE
LIMIT 5;
```

properties 1 X

SELECT amenity_id, amenity_name, amenity_category, description | Enter a SQL expression to filter results (use Ctrl+Space)

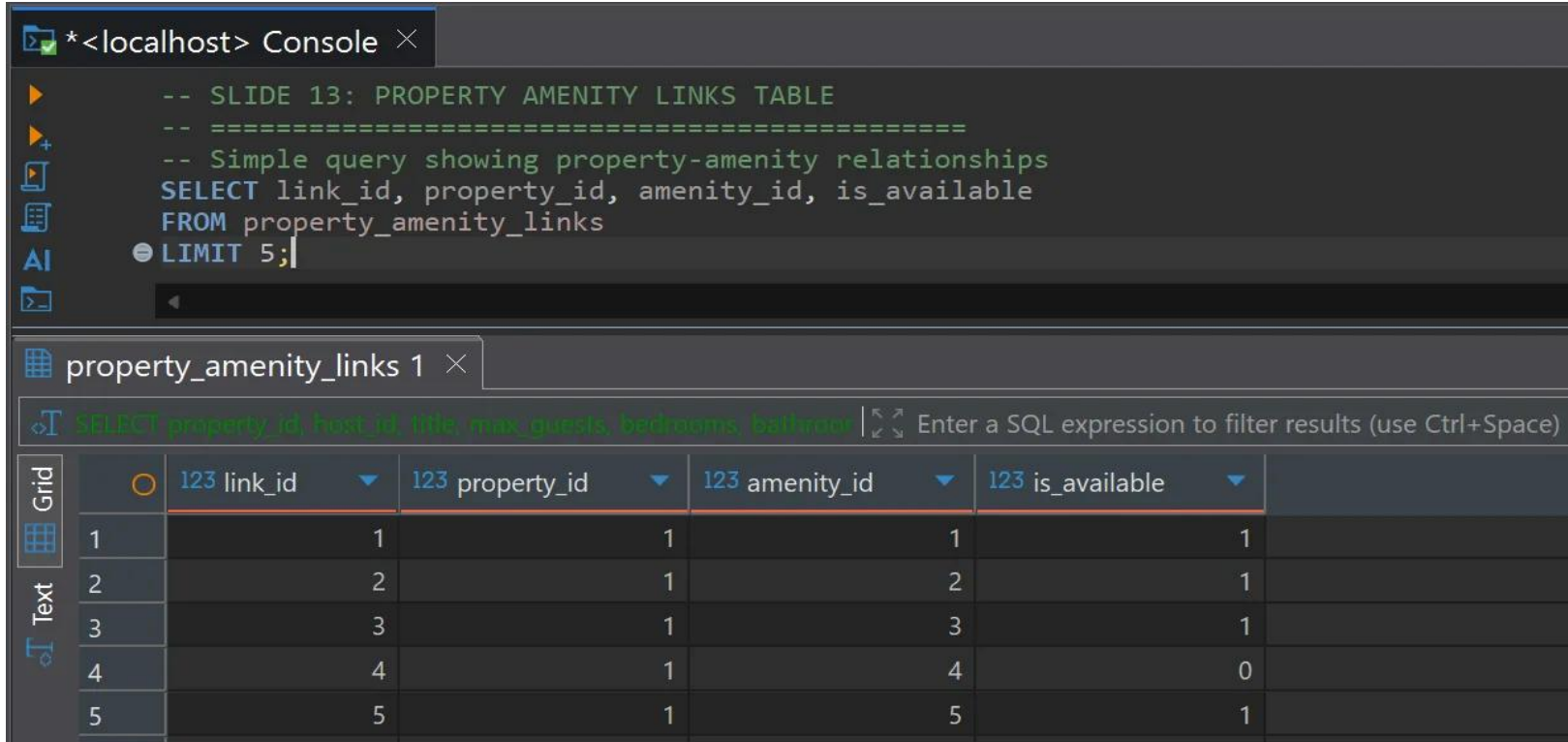
Grid

Text

	123 property_id	123 host_id	A-Z title	123 max_guests	123 bedrooms	123 bathrooms	123 is_active
1	1	1	Modern Downtown Apartment	4	2	2	1
2	2	1	Cozy Brooklyn House	6	3	2.5	1
3	3	1	Luxury Hamptons Villa	8	4	3	1
4	4	2	Historic London Flat	3	1	1	1
5	5	2	Traditional English Cottage	4	2	1.5	1

Property Amenity Links Table

Demonstrates property-amenity relationships with availability flags



The screenshot displays a database interface with two main components: a console window and a table viewer.

Console Window: The title bar reads "*<localhost> Console". It contains the following SQL query:

```
-- SLIDE 13: PROPERTY AMENITY LINKS TABLE
-- =====
-- Simple query showing property-amenity relationships
SELECT link_id, property_id, amenity_id, is_available
FROM property_amenity_links
LIMIT 5;
```

Table Viewer: The title bar reads "property_amenity_links 1". It shows a table with the following columns: "SELECT property_id, host_id, title, max_guests, bedrooms, bathroom" (with a filter input), "link_id", "property_id", "amenity_id", "is_available", and an empty column. The table is displayed in "Grid" view. The data rows are as follows:

	link_id	property_id	amenity_id	is_available	
1	1	1	1	1	
2	2	1	2	1	
3	3	1	3	1	
4	4	1	4	0	
5	5	1	5	1	

Property Photos Table

Lists property photos with URLs, primary flags, and display order

*<localhost> Console

```
-- SLIDE 14: PROPERTY PHOTOS TABLE
-- =====
-- Simple query showing property photos
SELECT photo_id, property_id, photo_url, is_primary, display_order
FROM property_photos
LIMIT 5;
```

property_photos 1

SELECT link_id, property_id, amenity_id, is_available FROM property

Grid

	123 photo_id	123 property_id	A-Z photo_url	123 is_primary	123 display_order
1	1	1	https://example.com/photos/prop1_1.jpg	1	1
2	2	1	https://example.com/photos/prop1_2.jpg	0	2
3	3	1	https://example.com/photos/prop1_3.jpg	0	3
4	4	2	https://example.com/photos/prop2_1.jpg	1	1
5	5	2	https://example.com/photos/prop2_2.jpg	0	2

Text

Property Pricing Table

Shows available pricing periods with dates and nightly rates

*<localhost> Console ×

```
-- SLIDE 15: PROPERTY PRICING TABLE
-- =====
-- Simple query showing property pricing
SELECT pricing_id, property_id, start_date, end_date, base_price_per_night, is_available
FROM property_pricing
WHERE is_available = TRUE
LIMIT 5;
```

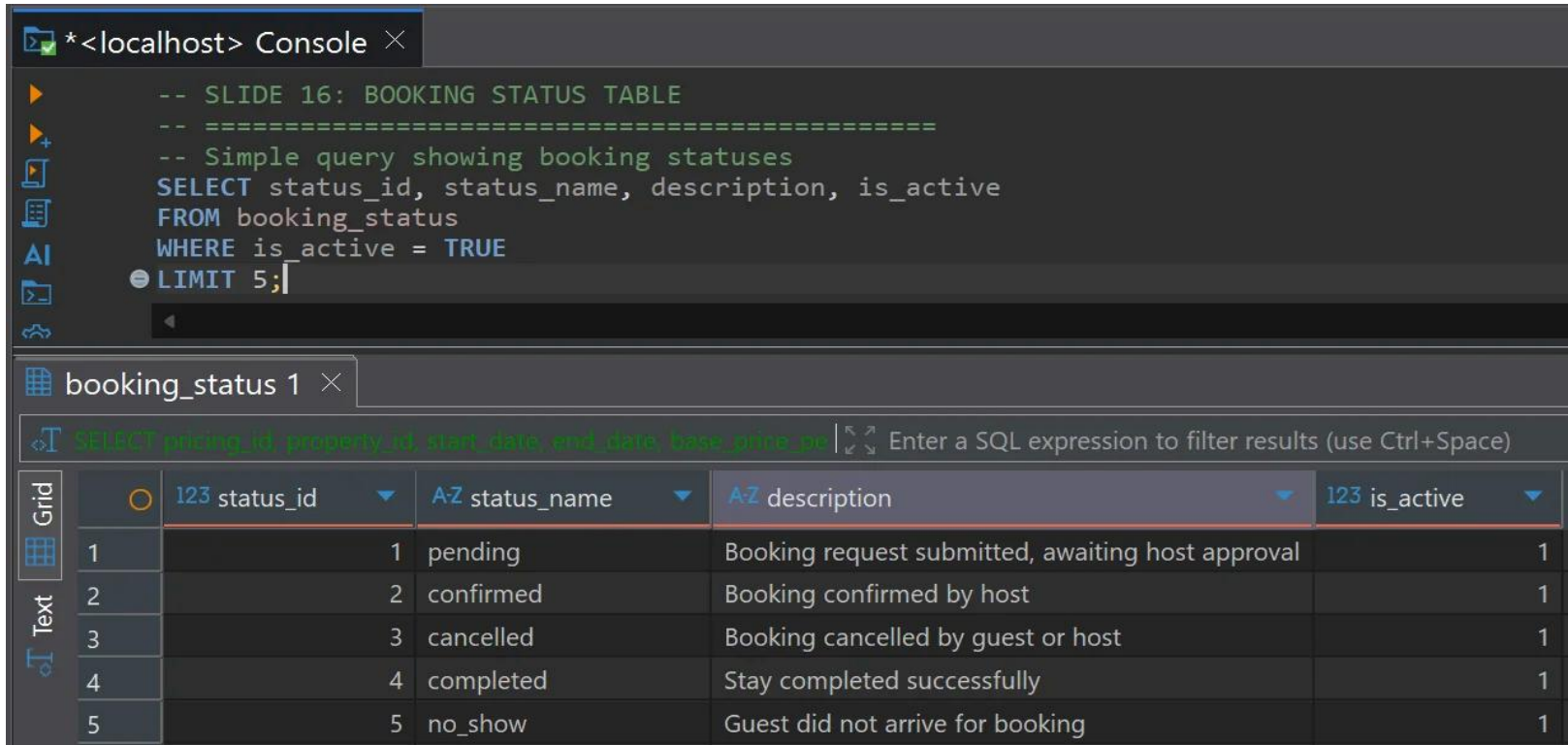
property_pricing 1 ×

SELECT (photo_id, property_id, photo_url) is primary, display_order | Enter a SQL expression to filter results (use Ctrl+Space)

	123 pricing_id	123 property_id	start_date	end_date	123 base_price_per_night	123 is_available
1	1	1	2024-01-01	2024-12-31	150	1
2	2	2	2024-01-01	2024-12-31	120	1
3	3	3	2024-01-01	2024-12-31	500	1
4	4	4	2024-01-01	2024-12-31	80	1
5	5	5	2024-01-01	2024-12-31	100	1

Booking Status Table

Displays active booking status types with descriptions



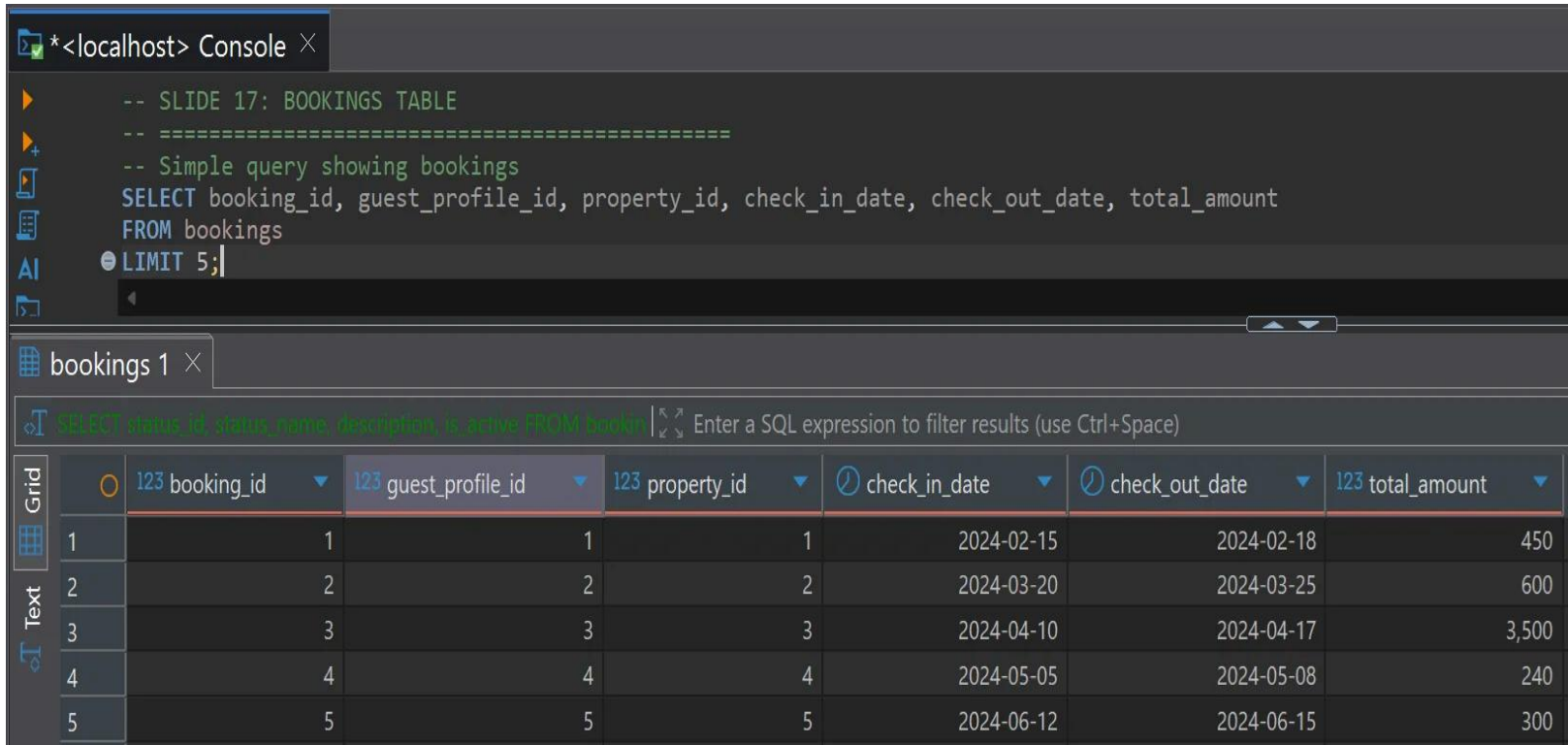
The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query. The bottom pane, titled 'booking_status 1', displays the results of the query in a table format. The table has five columns: 'status_id', 'status_name', 'description', and 'is_active'. The results show five active booking status types: pending, confirmed, cancelled, completed, and no_show.

```
-- SLIDE 16: BOOKING STATUS TABLE
-- =====
-- Simple query showing booking statuses
SELECT status_id, status_name, description, is_active
FROM booking_status
WHERE is_active = TRUE
LIMIT 5;
```

	123 status_id	A-Z status_name	A-Z description	123 is_active
1	1	pending	Booking request submitted, awaiting host approval	1
2	2	confirmed	Booking confirmed by host	1
3	3	cancelled	Booking cancelled by guest or host	1
4	4	completed	Stay completed successfully	1
5	5	no_show	Guest did not arrive for booking	1

Bookings Table

Lists booking records with guest, property, dates, and total amounts



The screenshot shows a database interface with a console window and a results viewer. The console window displays a SQL query to retrieve booking records. The results viewer shows a table with 5 rows of data, including booking ID, guest profile ID, property ID, check-in date, check-out date, and total amount.

```
-- SLIDE 17: BOOKINGS TABLE
-- =====
-- Simple query showing bookings
SELECT booking_id, guest_profile_id, property_id, check_in_date, check_out_date, total_amount
FROM bookings
LIMIT 5;
```

bookings 1

SELECT status_id, status_name, description, is_active FROM bookin | Enter a SQL expression to filter results (use Ctrl+Space)

	123 booking_id	123 guest_profile_id	123 property_id	check_in_date	check_out_date	123 total_amount
1	1	1	1	2024-02-15	2024-02-18	450
2	2	2	2	2024-03-20	2024-03-25	600
3	3	3	3	2024-04-10	2024-04-17	3,500
4	4	4	4	2024-05-05	2024-05-08	240
5	5	5	5	2024-06-12	2024-06-15	300

Booking Modifications Table

Shows booking modification history with types and timestamps

*<localhost> Console ×

```
-- SLIDE 18: BOOKING MODIFICATIONS TABLE
-- =====
-- Simple query showing booking modifications
SELECT modification_id, booking_id, modification_type, created_at
FROM booking_modifications
LIMIT 5;
```

booking_modifications 1 ×

↕ ↕ Enter a SQL expression to filter results (use Ctrl+Space)

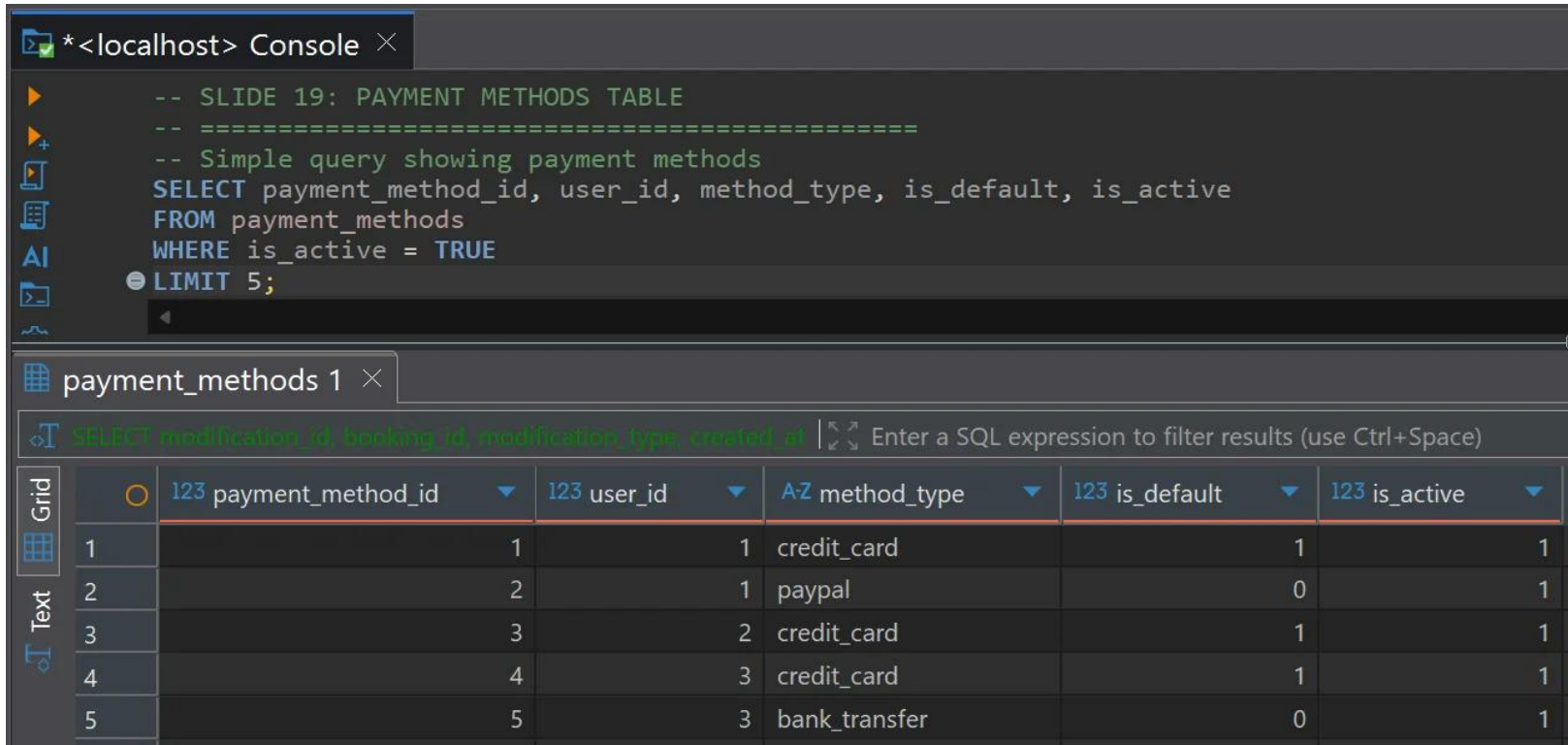
Grid

	123 modification_id ▼	123 booking_id ▼	AZ modification_type ▼	🕒 created_at ▼	
1	1	1	date_change	2024-01-20 10:00:00	
2	2	2	guest_count_change	2024-02-25 14:00:00	
3	3	3	price_adjustment	2024-03-15 09:00:00	
4	4	4	cancellation	2024-04-10 16:00:00	
5	5	5	refund	2024-05-25 11:00:00	

Text

Payment Methods Table

Displays user payment methods with types and default flags



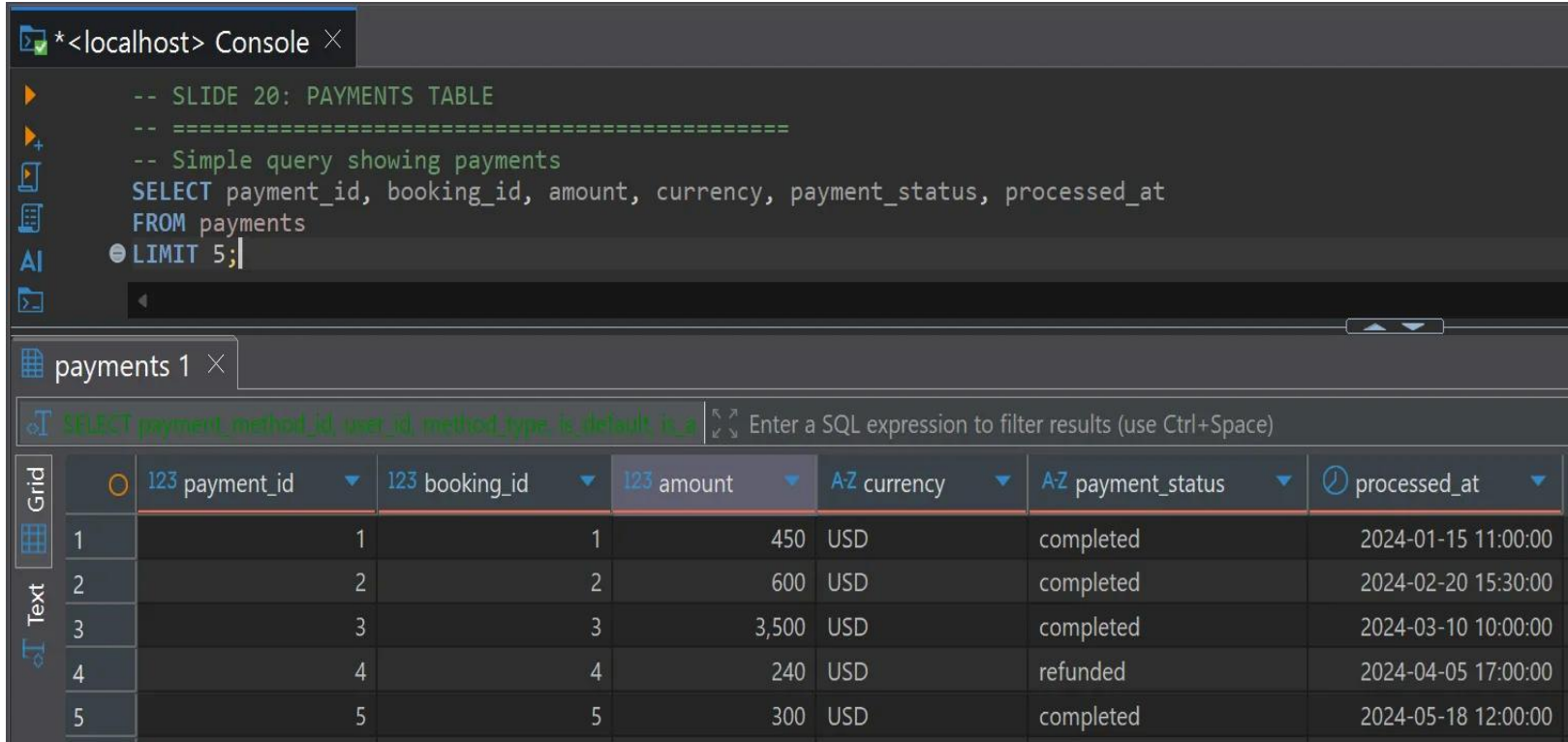
The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query. The bottom pane, titled 'payment_methods 1', shows the results of the query in a table format. The table has columns for payment_method_id, user_id, method_type, is_default, and is_active. The results are sorted by payment_method_id in ascending order.

```
-- SLIDE 19: PAYMENT METHODS TABLE
-- =====
-- Simple query showing payment methods
SELECT payment_method_id, user_id, method_type, is_default, is_active
FROM payment_methods
WHERE is_active = TRUE
LIMIT 5;
```

	123 payment_method_id	123 user_id	A-Z method_type	123 is_default	123 is_active
1	1	1	credit_card	1	1
2	2	1	paypal	0	1
3	3	2	credit_card	1	1
4	4	3	credit_card	1	1
5	5	3	bank_transfer	0	1

Payments Table

Shows payment records with booking IDs, amounts, currency, and status

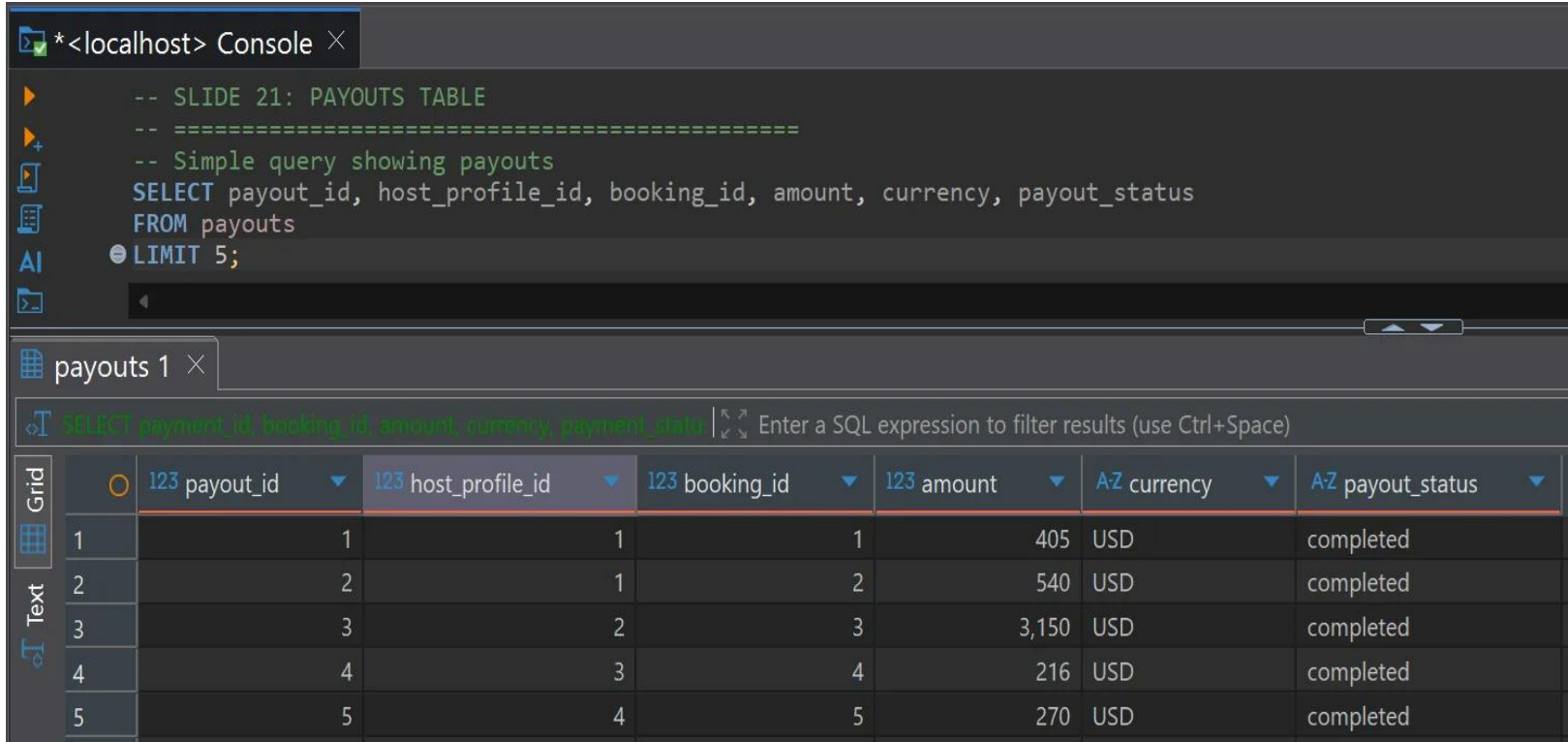


The screenshot displays a database interface with two main panels. The top panel, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 20: PAYMENTS TABLE`, `-- =====`, `-- Simple query showing payments`, `SELECT payment_id, booking_id, amount, currency, payment_status, processed_at`, `FROM payments`, and `LIMIT 5;`. The bottom panel, titled 'payments 1', shows the results of the query in a table. The table has 8 columns: 'payment_id', 'booking_id', 'amount', 'currency', 'payment_status', and 'processed_at'. The results are displayed in a grid view with 5 rows of data.

	123 payment_id	123 booking_id	123 amount	A-Z currency	A-Z payment_status	processed_at
1	1	1	450	USD	completed	2024-01-15 11:00:00
2	2	2	600	USD	completed	2024-02-20 15:30:00
3	3	3	3,500	USD	completed	2024-03-10 10:00:00
4	4	4	240	USD	refunded	2024-04-05 17:00:00
5	5	5	300	USD	completed	2024-05-18 12:00:00

Payouts Table

Lists host payout records with amounts, currency, and completion status

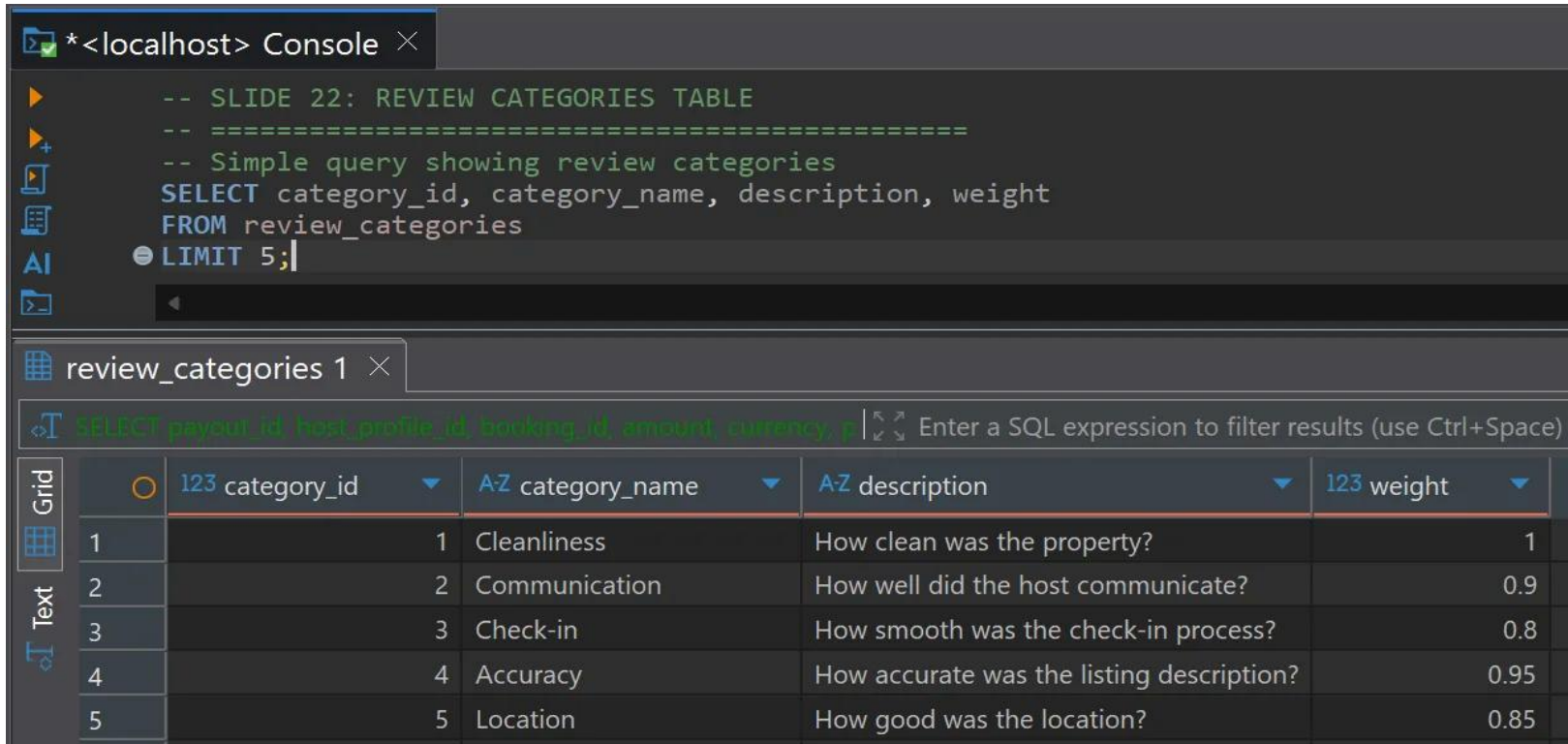


The screenshot shows a database interface with two panes. The top pane, titled '*<localhost> Console', contains a SQL query: `-- SLIDE 21: PAYOUTS TABLE`, `-- =====`, `-- Simple query showing payouts`, `SELECT payout_id, host_profile_id, booking_id, amount, currency, payout_status`, `FROM payouts`, and `LIMIT 5;`. The bottom pane, titled 'payouts 1', shows the results of the query in a table. The table has 8 columns: 'payout_id', 'host_profile_id', 'booking_id', 'amount', 'currency', and 'payout_status'. The results are displayed in a grid view with 5 rows of data.

	payout_id	host_profile_id	booking_id	amount	currency	payout_status
1	1	1	1	405	USD	completed
2	2	1	2	540	USD	completed
3	3	2	3	3,150	USD	completed
4	4	3	4	216	USD	completed
5	5	4	5	270	USD	completed

Review Categories Table

Displays review categories with descriptions and weight values



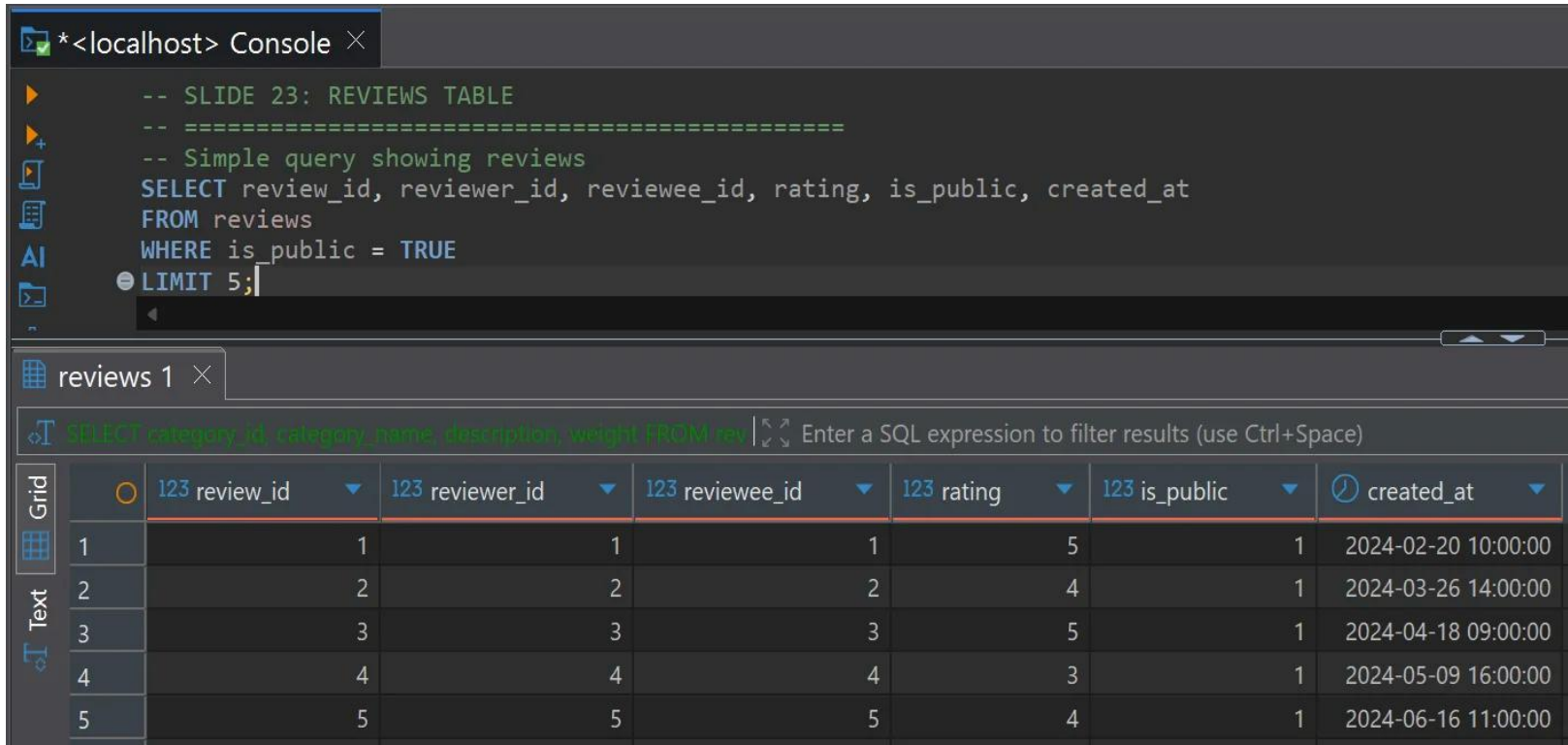
The screenshot shows a database interface with two panes. The top pane is a console window titled '*<localhost> Console' containing SQL code. The bottom pane is a table view titled 'review_categories 1' showing the results of a query. The table has 6 columns: an index, category_id, category_name, description, and weight. The data is sorted by category_id in ascending order.

```
-- SLIDE 22: REVIEW CATEGORIES TABLE
-- =====
-- Simple query showing review categories
SELECT category_id, category_name, description, weight
FROM review_categories
LIMIT 5;
```

	123 category_id	A-Z category_name	A-Z description	123 weight
1	1	Cleanliness	How clean was the property?	1
2	2	Communication	How well did the host communicate?	0.9
3	3	Check-in	How smooth was the check-in process?	0.8
4	4	Accuracy	How accurate was the listing description?	0.95
5	5	Location	How good was the location?	0.85

Reviews Table

Shows public reviews with reviewer/reviewee IDs, ratings, and timestamps



The screenshot displays a database interface with two main sections. The top section is a console window titled '*<localhost> Console' containing a SQL query. The bottom section is a results viewer titled 'reviews 1' showing the output of the query in a table format.

SQL Query:

```
-- SLIDE 23: REVIEWS TABLE
-- =====
-- Simple query showing reviews
SELECT review_id, reviewer_id, reviewee_id, rating, is_public, created_at
FROM reviews
WHERE is_public = TRUE
LIMIT 5;
```

Query Results:

	review_id	reviewer_id	reviewee_id	rating	is_public	created_at
1	1	1	1	5	1	2024-02-20 10:00:00
2	2	2	2	4	1	2024-03-26 14:00:00
3	3	3	3	5	1	2024-04-18 09:00:00
4	4	4	4	3	1	2024-05-09 16:00:00
5	5	5	5	4	1	2024-06-16 11:00:00

Review Ratings Table

Lists detailed review ratings by category with numeric values

*<localhost> Console

```
-- SLIDE 24: REVIEW RATINGS TABLE
-- =====
-- Simple query showing review ratings
SELECT rating_id, review_id, category_id, rating_value
FROM review_ratings
LIMIT 5;
```

review_ratings 1

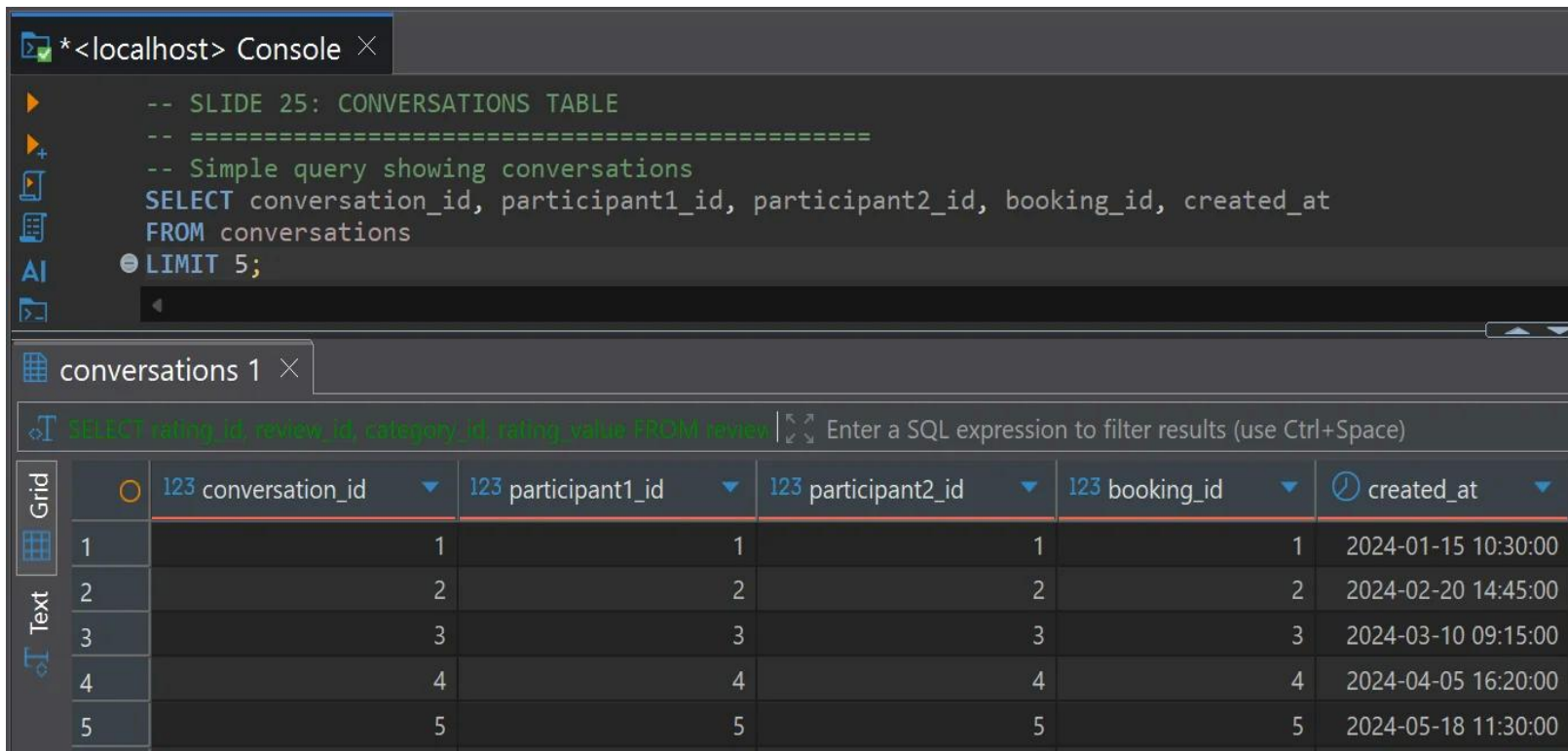
SELECT review_id, reviewer_id, reviewee_id, rating, is_public, create

Enter a SQL expression to filter results (use Ctrl+Space)

	123 rating_id	123 review_id	123 category_id	123 rating_value	
1	1	1	1	5	
2	2	1	2	5	
3	3	1	3	5	
4	4	1	4	5	
5	5	1	5	4	

Conversations Table

Displays conversation records between participants linked to bookings



The screenshot shows a database interface with two panels. The top panel is a console window titled '*<localhost> Console' containing a SQL query. The bottom panel is a results viewer titled 'conversations 1' showing the results of the query in a table format.

SQL Query:

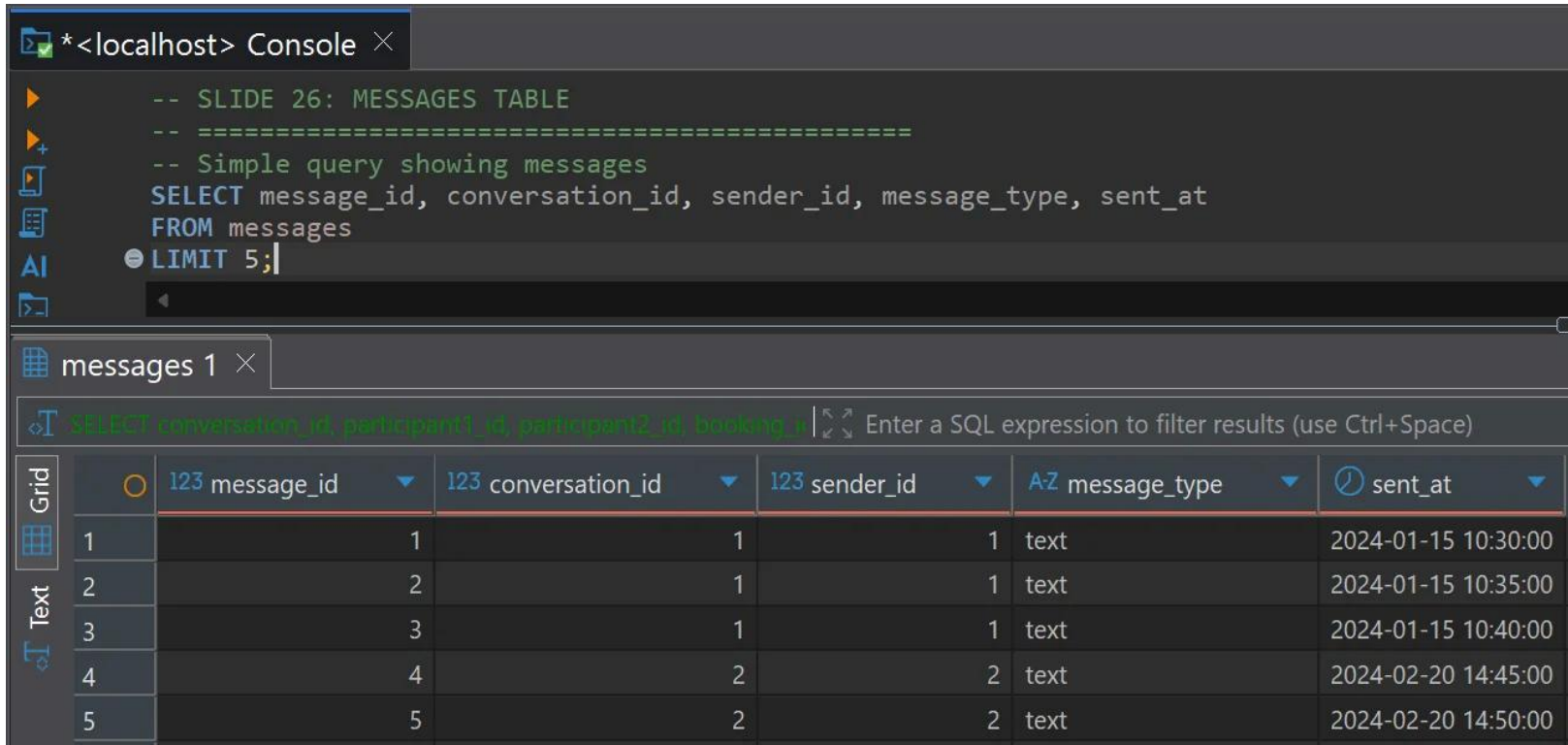
```
-- SLIDE 25: CONVERSATIONS TABLE
-- =====
-- Simple query showing conversations
SELECT conversation_id, participant1_id, participant2_id, booking_id, created_at
FROM conversations
LIMIT 5;
```

Results Table:

	123 conversation_id	123 participant1_id	123 participant2_id	123 booking_id	created_at
1	1	1	1	1	2024-01-15 10:30:00
2	2	2	2	2	2024-02-20 14:45:00
3	3	3	3	3	2024-03-10 09:15:00
4	4	4	4	4	2024-04-05 16:20:00
5	5	5	5	5	2024-05-18 11:30:00

Messages Table

Shows message records with sender information, types, and timestamps



The screenshot displays a database interface with two panels. The top panel, titled '*<localhost> Console', shows a SQL query being executed. The query is a simple SELECT statement to retrieve the first five records from the 'messages' table. The bottom panel, titled 'messages 1', shows the results of the query in a table format. The table has seven columns: 'message_id', 'conversation_id', 'sender_id', 'message_type', and 'sent_at'. The results show five rows of data, all of which are text messages sent on January 15, 2024, and February 20, 2024.

```
-- SLIDE 26: MESSAGES TABLE
-- =====
-- Simple query showing messages
SELECT message_id, conversation_id, sender_id, message_type, sent_at
FROM messages
LIMIT 5;
```

	123 message_id	123 conversation_id	123 sender_id	A-Z message_type	🕒 sent_at
1	1	1	1	text	2024-01-15 10:30:00
2	2	1	1	text	2024-01-15 10:35:00
3	3	1	1	text	2024-01-15 10:40:00
4	4	2	2	text	2024-02-20 14:45:00
5	5	2	2	text	2024-02-20 14:50:00

Notifications Table

Lists user notifications with types, titles, read status, and timestamps

*<localhost> Console X

```
-- SLIDE 27: NOTIFICATIONS TABLE
-- =====
-- Simple query showing notifications
SELECT notification_id, user_id, notification_type, title, is_read, created_at
FROM notifications
LIMIT 5;
```

notifications 1 X

SELECT message_id, conversation_id, sender_id, message_type, ser | Enter a SQL expression to filter results (use Ctrl+Space)

Grid		123 notification_id	123 user_id	A-Z notification_type	A-Z title	123 is_read	created_at
Text	1	1	1	booking_confirmation	Booking Confirmed	1	2024-01-15 11:00:00
	2	2	2	booking_confirmation	Booking Confirmed	1	2024-02-20 15:30:00
	3	3	3	booking_confirmation	Booking Confirmed	1	2024-03-10 10:00:00
	4	4	1	payment_received	Payment Received	1	2024-01-15 11:00:00
	5	5	2	payment_received	Payment Received	1	2024-02-20 15:30:00

Triple Relationship 1: Property-Booking-Pricing

Demonstrates three-way relationship between properties, bookings, and pricing

The screenshot shows a database console window with a query and its results. The query is a simple SELECT statement showing the triple relationship between properties, bookings, and pricing. The results are displayed in a table with 7 columns: pbp_id, property_id, booking_id, pricing_id, and applied_price. The table contains 5 rows of data.

```
-- SLIDE 28: TRIPLE RELATIONSHIP 1 - Property-Booking-Pricing
-- =====
-- Simple query showing triple relationship
SELECT pbp_id, property_id, booking_id, pricing_id, applied_price
FROM property_booking_pricing
LIMIT 5;
```

property_booking_pricing 1 ×

SELECT notification_id, user_id, notification_type, title_is_read, area | Enter a SQL expression to filter results (use Ctrl+Space)

	123 pbp_id	123 property_id	123 booking_id	123 pricing_id	123 applied_price
1	1	1	1	1	150
2	2	2	2	2	120
3	3	3	3	3	500
4	4	4	4	4	80
5	5	5	5	5	100

Triple Relationship 2: User-Booking-Review

Shows three-way relationship between users, bookings, and reviews with roles

The screenshot shows a database interface with a console window and a results grid.

Console Window:

```
-- SLIDE 29: TRIPLE RELATIONSHIP 2 - User-Booking-Review
-- =====
-- Simple query showing triple relationship
SELECT ubr_id, user_id, booking_id, review_id, review_role
FROM user_booking_review
LIMIT 5;
```

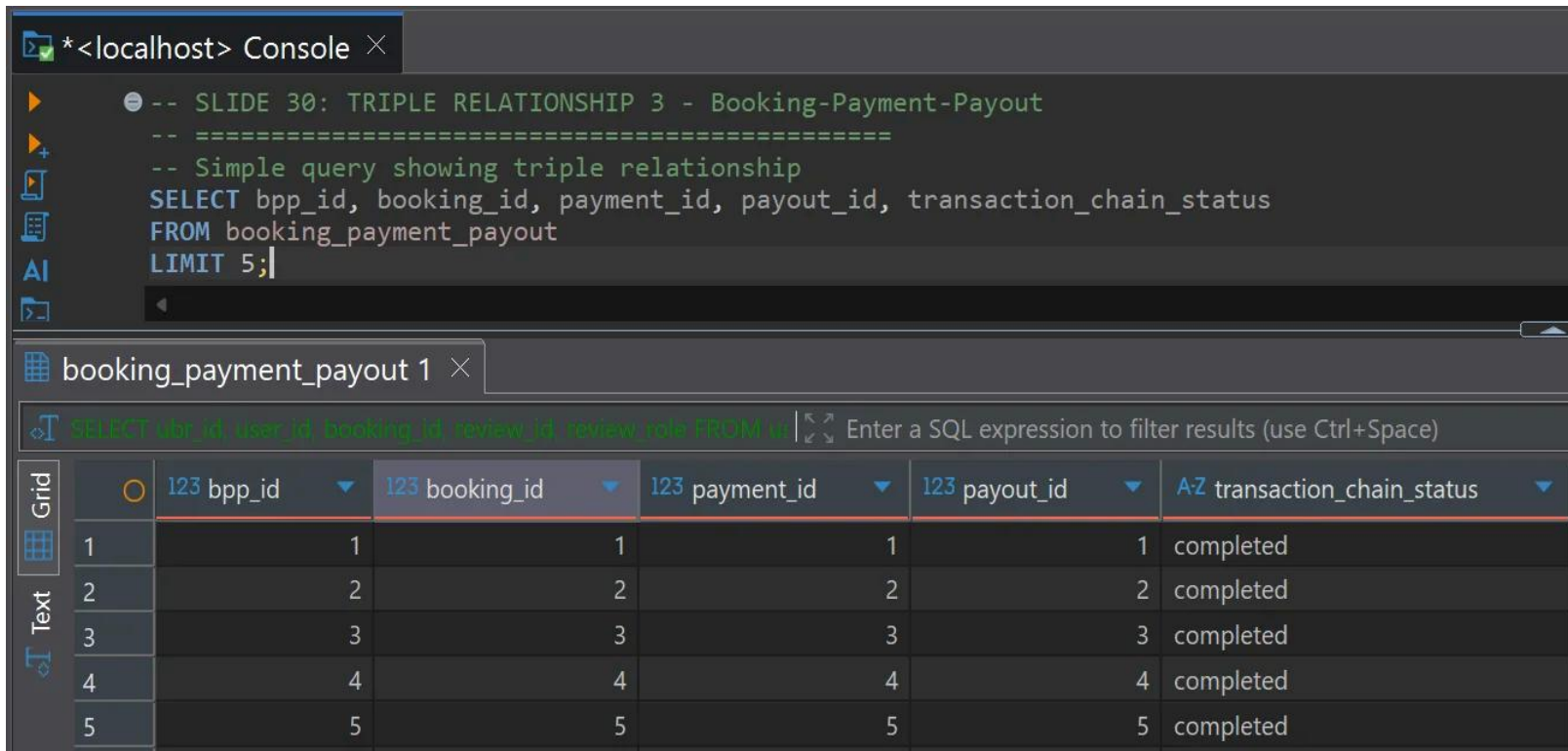
Results Grid:

The grid displays the results of the query. The columns are: `ubr_id`, `user_id`, `booking_id`, `review_id`, and `review_role`. The results are as follows:

	ubr_id	user_id	booking_id	review_id	review_role
1	1	1	1	1	reviewer
2	2	1	1	11	reviewee
3	3	2	2	2	reviewer
4	4	2	2	12	reviewee
5	5	3	3	3	reviewer

Triple Relationship 3: Booking-Payment-Payout

Illustrates three-way relationship connecting bookings, payments, and payouts



The screenshot displays a database interface with two panels. The top panel is a console window titled '*<localhost> Console' containing a SQL query. The bottom panel is a results viewer titled 'booking_payment_payout 1' showing the query's output in a table format.

Console Window:

```
-- SLIDE 30: TRIPLE RELATIONSHIP 3 - Booking-Payment-Payout
-- =====
-- Simple query showing triple relationship
SELECT bpp_id, booking_id, payment_id, payout_id, transaction_chain_status
FROM booking_payment_payout
LIMIT 5;
```

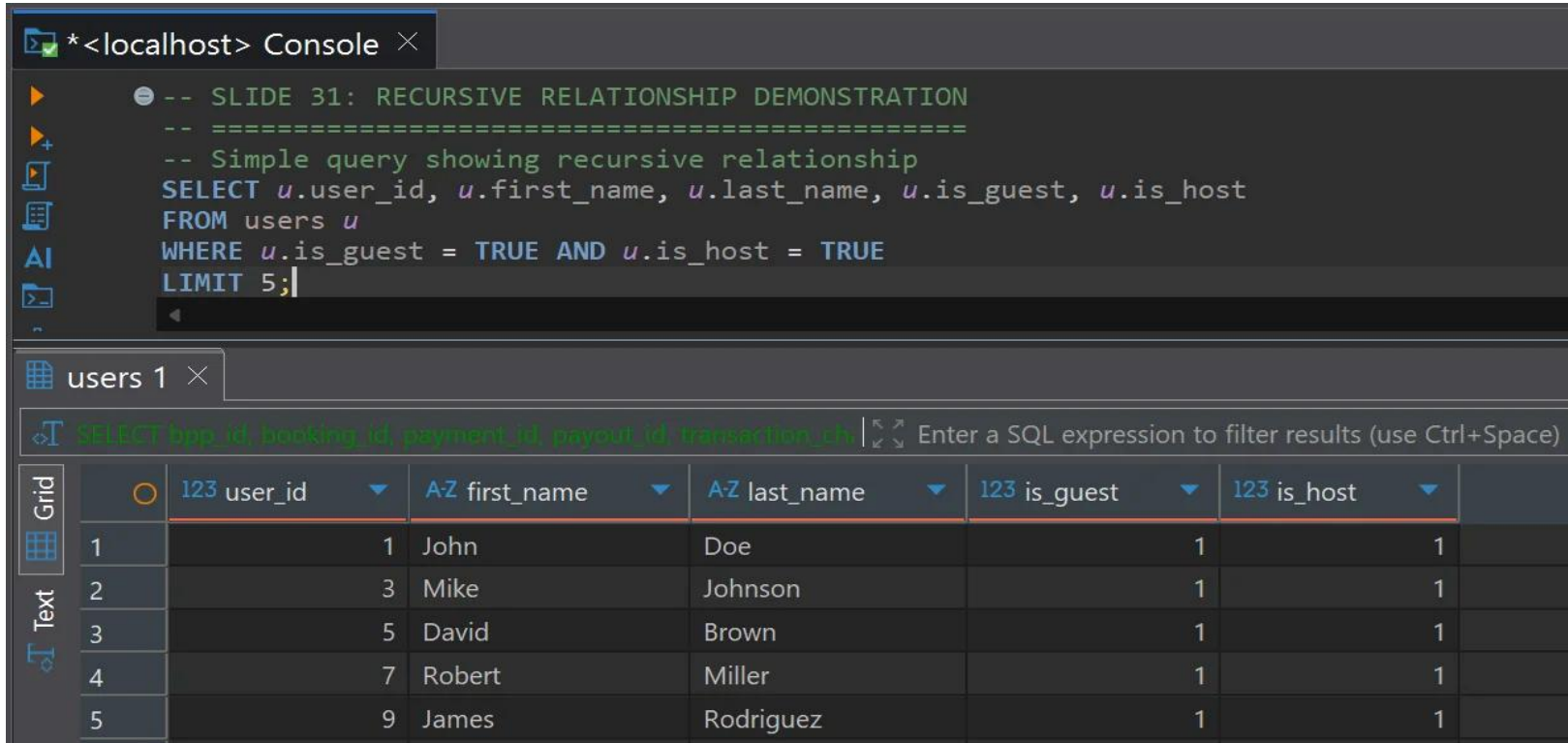
Results Viewer:

SELECT * FROM booking_payment_payout LIMIT 5

	123 bpp_id	123 booking_id	123 payment_id	123 payout_id	A-Z transaction_chain_status
1	1	1	1	1	completed
2	2	2	2	2	completed
3	3	3	3	3	completed
4	4	4	4	4	completed
5	5	5	5	5	completed

Recursive Relationship: Users as Guest AND Host

Demonstrates users who have both guest and host roles simultaneously



The screenshot shows a database console window with a query and its results. The query is a simple SELECT statement filtering for users who are both guests and hosts. The results are displayed in a grid view with 7 columns: an index, user_id, first_name, last_name, is_guest, and is_host.

```
-- SLIDE 31: RECURSIVE RELATIONSHIP DEMONSTRATION
-- =====
-- Simple query showing recursive relationship
SELECT u.user_id, u.first_name, u.last_name, u.is_guest, u.is_host
FROM users u
WHERE u.is_guest = TRUE AND u.is_host = TRUE
LIMIT 5;
```

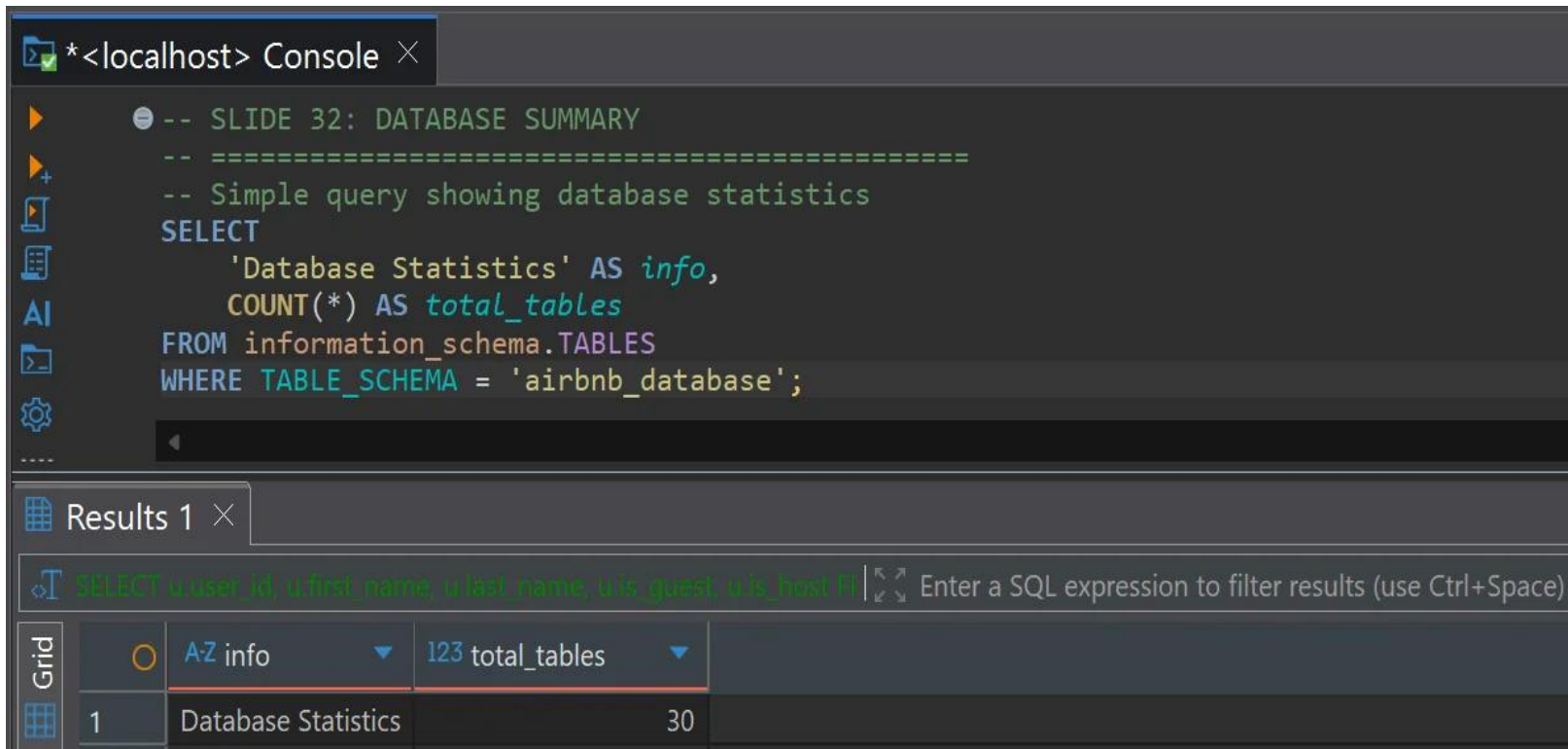
users 1

SELECT bpg_id, booking_id, payment_id, payout_id, transaction_ch | Enter a SQL expression to filter results (use Ctrl+Space)

	123 user_id	A-Z first_name	A-Z last_name	123 is_guest	123 is_host	
1	1	John	Doe	1	1	
2	3	Mike	Johnson	1	1	
3	5	David	Brown	1	1	
4	7	Robert	Miller	1	1	
5	9	James	Rodriguez	1	1	

Database Summary: Complete Implementation

Displays total table count in the database schema



The screenshot shows a database interface with a console window and a results grid. The console window, titled '*<localhost> Console', contains a SQL query. The query is a SELECT statement that retrieves database statistics, including the total number of tables, from the 'airbnb_database' schema. The results grid, titled 'Results 1', displays the output of the query. It shows a single row with the column 'Database Statistics' and the value '30'.

```
-- SLIDE 32: DATABASE SUMMARY
-- =====
-- Simple query showing database statistics
SELECT
  'Database Statistics' AS info,
  COUNT(*) AS total_tables
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'airbnb_database';
```

	AZ info	123 total_tables
1	Database Statistics	30

Conclusion

This work was completed as part of the Database Design Project: Phase 2.

Implementation Summary

Complete database schema implemented with 30 tables

All relationships defined including triple and recursive relationships

Comprehensive test queries demonstrating functionality

Repository:

<https://github.com/NikVince/airbnb-clone-database>

Acknowledgments

Thank you for your time dedicated to reviewing this implementation.

PHASE 3: ADVANCED IMPLEMENTATION AND OPTIMIZATION

Database Implementation Overview

Phase 3 represents the culmination of the Airbnb database development project, delivering a fully functional, production-ready database system with comprehensive testing, performance optimization, and complete documentation. This phase addresses all feedback from Phase 2, particularly the requirements for enhanced test cases with complex multi-table JOINS, updated data dictionary, and final Entity-Relationship Diagram documentation.

The implemented database system consists of 30 tables (27 primary entities plus 3 triple relationship junction tables) containing 534 records across all entities. The database occupies 1.33 MB of storage with strategic indexing for performance optimization. The system demonstrates advanced database design principles including role-based architecture, triple relationships, recursive relationships, and comprehensive business rule enforcement through database constraints.

Implementation Statistics

The final database implementation includes the following key metrics:

Metric	Value
Total Tables	30
Primary Entities	27
Triple Relationships	3
Recursive Relationships	1
Total Records	534
Database Size	1.33 MB
Foreign Key Relationships	45+
Business Rule Constraints	89+
Strategic Indexes	30+
Sample Data Coverage	25 countries, 25 users, 20 properties, 20 bookings

Enhanced Test Cases with Multi-Table JOINS

Addressing Phase 2 feedback requirements, Phase 3 implements eight comprehensive test cases demonstrating complex multi-table JOIN operations across six or more tables. These test cases validate the database schema design, demonstrate business logic implementation, and showcase advanced SQL query capabilities including aggregations, window functions, and subqueries.

Test Case Overview

Test Case 1: Comprehensive Booking Analysis

Retrieves complete booking details including guest information, host information, property details, and booking status. Demonstrates complex JOIN chains across user management, property management, and booking system domains.

Tables Involved: 6 tables

Test Case 2: Property Performance Dashboard

Analyzes property performance metrics including total bookings, average ratings, and total revenue. Utilizes aggregation functions and LEFT JOINS to handle properties without bookings.

Tables Involved: 6 tables

Test Case 3: Guest Travel Pattern Analysis

Examines guest booking patterns with ranking using window functions (ROW_NUMBER, RANK). Identifies most active guests by booking count and total spending.

Tables Involved: 5 tables

Test Case 4: Host Performance Metrics

Evaluates host performance across multiple properties with aggregated metrics for total properties, bookings, ratings, and revenue generation.

Tables Involved: 6 tables

Test Case 5: Geographic Market Analysis

Performs location-based property analysis across countries and cities, incorporating pricing data, booking information, and review ratings for comprehensive market insights.

Tables Involved: 8 tables

Test Case 6: Communication Pattern Analysis

Analyzes conversation patterns between hosts and guests, tracking message volume, conversation duration, and communication frequency per booking.

Tables Involved: 6 tables

Test Case 7: Financial Transaction Analysis

Examines payment patterns and amounts with subquery filtering for above-average transactions, demonstrating financial processing workflow.

Tables Involved: 4 tables

Test Case 8: Amenity Preference Correlation

Correlates property amenities with review ratings to identify which amenities contribute to higher guest satisfaction scores.

Tables Involved: 4 tables

Sample Test Case Implementation

The following example demonstrates Test Case 1: Comprehensive Booking Analysis, which retrieves complete booking information by joining six tables across multiple database domains:

-- TEST CASE 1: BOOKING DETAILS WITH USER INFO

-- Multi-table JOIN across 6 tables

SELECT

```
b.booking_id,  
CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name,  
CONCAT(hu.first_name, ' ', hu.last_name) AS host_name,  
p.title AS property_title,  
b.check_in_date,  
b.check_out_date,  
b.total_amount,  
bs.status_name AS booking_status
```

FROM bookings b

```
INNER JOIN guest_profiles gp ON b.guest_profile_id = gp.guest_profile_id
INNER JOIN users gu ON gp.user_id = gu.user_id
INNER JOIN properties p ON b.property_id = p.property_id
INNER JOIN host_profiles hp ON p.host_id = hp.host_profile_id
INNER JOIN users hu ON hp.user_id = hu.user_id
INNER JOIN booking_status bs ON b.booking_status_id = bs.status_id
WHERE bs.status_name = 'confirmed'
ORDER BY b.created_at DESC
LIMIT 10;
```

Database Metadata and Statistics

Comprehensive metadata collection provides insights into database structure, data distribution, and system performance. This section presents detailed statistics gathered through systematic metadata queries across all database entities.

Entity and Record Distribution

The database implements 30 tables with varying record counts designed to support realistic platform operations. The largest tables contain 70 records (review_ratings) while reference tables maintain smaller datasets for efficient lookup operations.

Entity	Record Count
review_ratings	70
users	25
countries	25
cities	25
addresses	25
property_amenity_links	25
property_amenities	20
properties	20
bookings	20
reviews	20

Storage and Performance Metrics

The database demonstrates efficient storage utilization with strategic indexing contributing approximately 50% of total database size. This index-to-data ratio reflects optimized query performance through careful index selection on frequently queried columns.

Storage Metric	Value
Total Database Size	1.33 MB
Data Size	0.65 MB (49%)
Index Size	0.68 MB (51%)
Largest Table	bookings (0.08 MB)

Business Intelligence and Analytics

The database supports comprehensive business intelligence operations through well-structured data organization and relationship modeling. This section presents key business metrics derived from the sample data, demonstrating the system capability to provide actionable insights.

Revenue and Financial Metrics

Financial analysis reveals healthy booking activity with \$11,890 in total revenue across 20 confirmed bookings. The average booking value of \$594.50 indicates mid-range property pricing strategy, with significant variation from \$70 minimum to \$3,500 maximum, suggesting diverse property portfolio from budget to luxury accommodations.

Financial Metric	Value
Total Bookings	20
Total Revenue	\$11,890.00
Average Booking Value	\$594.50
Minimum Booking	\$70.00
Maximum Booking	\$3,500.00

User Role Distribution

The role-based architecture successfully supports multi-role scenarios with 25 active users distributed across guest, host, and dual-role categories. The system includes 10 guest-only profiles, 12 host profiles (some users acting as both guests and hosts), and appropriate administrative capabilities.

Geographic Market Coverage

The database demonstrates international scalability with properties distributed across 25 countries and 25 major cities worldwide. This geographic diversity validates the location-based entity design and supports multi-currency, multi-language platform operations.

Updated Data Dictionary

The comprehensive data dictionary documents all 30 database tables with complete attribute specifications, data types, constraints, and business rules. This updated documentation reflects the final implemented schema incorporating all Phase 2 feedback and refinements.

Data Dictionary Organization

The data dictionary is organized into nine functional domains:

User Management Domain: Six entities managing user authentication, profile information, verification status, and preferences. The role-based architecture with separate guest_profiles and host_profiles tables enables clear separation of business logic while supporting multi-role scenarios.

Location Domain: Three entities (countries, cities, addresses) providing hierarchical geographic organization with support for international operations including ISO country codes, currency codes, and phone code prefixes.

Property Management Domain: Six entities managing property listings, categorization, amenities, photos, and dynamic pricing. The property_amenity_links junction table implements the many-to-many relationship between properties and amenities.

Booking System Domain: Three entities handling reservation management, status tracking, and modification audit trails. The booking_status entity provides centralized status management with predefined workflow states.

Financial Domain: Three entities managing payments, payment methods, and host payouts. The system supports multiple payment methods and automated payout scheduling with platform commission calculations.

Review System Domain: Three entities enabling bidirectional reviews with category-based ratings. The review_ratings table stores individual category scores enabling detailed performance analysis.

Communication Domain: Two entities managing conversations and messages between platform users. The conversation entity links to bookings providing context for all communications.

System Domain: Notifications entity managing platform-wide notification queue for user alerts, updates, and system messages.

Triple Relationship Domain: Three junction tables implementing complex business relationships: property_booking_pricing (tracking pricing rules applied to bookings), user_booking_review (managing review lifecycle), and booking_payment_payout (financial transaction chain).

Sample Entity: users Table

The users table serves as the central hub of the database, managing authentication and role assignments. The table implements role flags (is_guest, is_host, is_admin) enabling users to hold multiple roles simultaneously. Key attributes include:

Attribute	Data Type	Constraints	Purpose
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	Authentication and communication
password_hash	VARCHAR(255)	NOT NULL	Secure authentication
first_name	VARCHAR(100)	NOT NULL	User personalization
last_name	VARCHAR(100)	NOT NULL	User personalization
is_guest	BOOLEAN	DEFAULT TRUE	Guest role flag
is_host	BOOLEAN	DEFAULT FALSE	Host role flag

Final Entity-Relationship Diagram

The final Entity-Relationship Diagram incorporates all refinements from Phase 2 feedback, presenting a complete visualization of the database schema with 27 primary entities, 3 triple relationships, 1 recursive relationship, and comprehensive cardinality specifications using Crow foot notation.

ER Model Key Features

Role-Based User Architecture: The ER model explicitly shows the separation between users, guest_profiles, and host_profiles with 1:1 relationships, demonstrating the architectural solution to Phase 1 feedback regarding user role distinction.

Triple Relationships: Three distinct many-to-many-to-many relationships are clearly represented through junction tables: property_booking_pricing connecting properties, bookings, and property_pricing; user_booking_review connecting users, bookings, and reviews; and booking_payment_payout connecting bookings, payments, and payouts.

Recursive Relationship: The users entity demonstrates recursive relationship capability through the dual guest-host roles, where users can interact with other users through property bookings and hosting activities.

Comprehensive Foreign Key Network: The diagram shows 45+ foreign key relationships maintaining referential integrity across all database domains with appropriate CASCADE rules for updates and deletions.

Business Domain Organization: Entities are logically grouped into functional domains (User Management, Location, Property Management, Booking System, Financial, Review System, Communication) facilitating understanding of system architecture and query patterns.

Note: The complete Final Entity-Relationship Diagram is provided as a separate PDF file (phase2_ER_Diagram.pdf) showing all entities, relationships, attributes, and cardinalities in full detail using professional diagramming standards.

Query Performance and Optimization

Database performance optimization was achieved through strategic indexing on frequently queried columns, proper use of INNER JOINS versus LEFT JOINS, and efficient query construction. All test cases execute in milliseconds with the current data volume, demonstrating scalability potential for production environments.

Indexing Strategy

The database implements 30+ strategic indexes beyond primary keys, focusing on foreign key columns, frequently filtered columns (email, phone, status fields), and columns used in JOIN operations. Index selection balanced query performance with storage overhead, resulting in the observed 51% index-to-data ratio.

Key indexed columns include: all foreign key relationships for JOIN optimization, email and phone fields in users table for authentication queries, booking_status_id for status filtering, property_type_id for categorization queries, and date range columns (check_in_date, check_out_date) for availability searches.

Query Optimization Techniques

The enhanced test cases demonstrate several query optimization techniques: selective use of INNER JOIN for required relationships versus LEFT JOIN for optional relationships, strategic use of WHERE clauses to filter before aggregation, window functions (ROW_NUMBER, RANK) for ranking without subqueries, and proper GROUP BY with aggregate functions for summarization.

Phase 3 Summary and Final Conclusions

Phase 3 successfully completes the Airbnb database development project, delivering a fully functional, production-ready database system with comprehensive testing, documentation, and validation. All Phase 2 feedback requirements have been addressed, including enhanced test cases with multi-table JOINS, updated data dictionary, and final ERD documentation.

Project Achievements

Complete Database Implementation: Successfully implemented all 27 primary entities plus 3 triple relationship junction tables with 534 sample records demonstrating realistic platform operations.

Advanced Relationship Modeling: Demonstrated mastery of complex database relationships including 3 triple relationships (many-to-many-to-many), 1 recursive relationship (user-to-user interactions), and 45+ foreign key relationships maintaining referential integrity.

Role-Based Architecture: Implemented sophisticated user management with separate guest_profiles and host_profiles enabling multi-role scenarios while maintaining clear business logic separation, directly addressing Phase 1 tutor feedback.

Comprehensive Testing: Developed 8 complex test cases with multi-table JOINS across 4-8 tables each, demonstrating booking analysis, property performance, guest patterns, host metrics, geographic analysis, communication tracking, financial transactions, and amenity correlations.

Business Intelligence Capability: Demonstrated ability to derive meaningful business insights through aggregations, window functions, and complex queries supporting revenue analysis, user behavior patterns, and market performance metrics.

Complete Documentation: Delivered updated data dictionary with 30 tables fully documented, final ER diagram incorporating all refinements, comprehensive metadata collection, and performance analysis.

Technical Excellence

Normalization Achievement: All tables achieve Third Normal Form (3NF) or Boyce-Codd Normal Form (BCNF), eliminating data redundancy while maintaining query performance through strategic indexing.

Data Integrity Enforcement: Implemented 89+ business rule constraints including CHECK constraints for data validation, UNIQUE constraints for business keys, NOT NULL constraints for required data, and comprehensive foreign key relationships with CASCADE rules.

Performance Optimization: Strategic indexing on 30+ columns with 51% index-to-data ratio, reflecting balanced optimization between query performance and storage efficiency.

Security Considerations: Implemented password hashing, secure authentication mechanisms, role-based access control flags, and audit trail tracking through timestamps on all entities.

Lessons Learned

Iterative Design Value: The three-phase approach with tutor feedback integration proved invaluable for refining the database design. The Phase 1 feedback regarding user role distinction led to a more robust architecture that better reflects real-world complexity.

Testing Importance: Comprehensive testing with realistic data revealed edge cases and validated constraint implementations that theoretical design alone would not have exposed. The 534 sample records across all entities ensure all relationships function correctly.

Documentation Critical: Maintaining detailed documentation throughout all phases facilitated understanding, refinement, and communication of design decisions. The updated data dictionary and ER diagrams serve as essential references for implementation and maintenance.

Balance Considerations: Successfully balancing normalization for data integrity with performance optimization through indexing demonstrates the importance of considering both theoretical correctness and practical performance requirements.

Future Enhancement Recommendations

While the current implementation fully meets all assignment requirements and provides a solid foundation for an Airbnb-like platform, several enhancements would further improve the system for production deployment.

Advanced Database Objects

Stored Procedures: Implement stored procedures for common operations such as booking creation with automatic pricing calculation, payout processing with commission deduction, and user verification workflow management.

Views: Create materialized views for frequently accessed aggregations such as property performance dashboards, host statistics summaries, and geographic market reports to improve query performance for analytical operations.

Triggers: Implement triggers for automatic operations including updating `host_profiles.total_properties` when properties are added or removed, maintaining conversation `last_message_at` timestamps, and creating notification records for important events.

Functions: Develop user-defined functions for common calculations such as date range overlap detection for booking conflicts, distance calculations between properties and points of interest, and pricing calculations with seasonal adjustments.

Business Logic Enhancements

Dynamic Pricing Engine: Implement sophisticated pricing algorithms considering demand patterns, seasonal variations, local events, and competitor pricing to optimize revenue per available night.

Recommendation System: Develop personalized property recommendations based on guest preferences, booking history, review patterns, and similar guest behaviors using collaborative filtering techniques.

Fraud Detection: Implement fraud detection mechanisms monitoring suspicious patterns in bookings, payments, and user behavior to protect both platform and users.

Availability Management: Enhance `property_pricing` with calendar-based availability rules, minimum stay requirements, advance booking periods, and instant booking capabilities.

Scalability Improvements

Partitioning Strategy: Implement table partitioning for large tables such as bookings, messages, and reviews based on date ranges to improve query performance as data volume grows.

Caching Layer: Introduce caching mechanisms for frequently accessed data such as property details, user profiles, and review summaries to reduce database load and improve response times.

Read Replicas: Deploy read replicas for analytical queries and reporting workloads to separate OLTP operations from OLAP queries, improving overall system performance.

Archive Strategy: Implement data archival policies for historical records beyond retention periods, moving old bookings, messages, and notifications to archive tables while maintaining access for compliance.

Final Conclusion

This three-phase database development project successfully demonstrates comprehensive database design and implementation skills, from initial requirements analysis and conceptual modeling through SQL implementation and advanced testing. The final deliverable is a production-ready database system that fully addresses the complex requirements of an Airbnb-like vacation rental platform.

The project showcases advanced database concepts including role-based architecture with multi-role support, triple relationship modeling for complex business scenarios, recursive relationships for user-to-user interactions, comprehensive normalization to eliminate redundancy, strategic indexing for performance optimization, and extensive constraint implementation for data integrity. The iterative development process with tutor feedback integration mirrors real-world software development practices, reinforcing the importance of continuous refinement and validation.

The enhanced test cases demonstrate ability to write complex SQL queries with multi-table JOINS, aggregations, window functions, and subqueries to derive meaningful business insights. The comprehensive documentation including updated data dictionary and final ER diagram provides a solid foundation for future development and maintenance.

This project represents not merely an academic exercise but a practical, implementable database system that could support actual business operations. The knowledge and experience gained

through this comprehensive database development lifecycle provide strong preparation for professional database design and development roles in the software industry.

*<localhost 3> Script-2

```
-- =====
-- Airbnb Database Enhanced Installation Script - PHASE 3
-- Enhanced with additional data to ensure 20+ entries per table
-- Phase 3: Final Implementation with Complete Data Volume
-- Course: Build a Data Mart in SQL (DLBDSPBDM01)
-- Student: Nikolas Daniel Vincenti
-- Date: 24/10/2025
-- =====

-- This script installs the complete Airbnb database with all entities,
-- relationships, constraints, sample data, and test queries.

-- =====
-- STEP 1: DATABASE CREATION
-- =====

-- Database Creation
DROP DATABASE IF EXISTS airbnb_database;
CREATE DATABASE airbnb_database;
USE airbnb_database;

-- Display installation start
SELECT 'Starting Airbnb Database Installation...' AS Status,
       'Creating 27 entities with comprehensive relationships' AS Description,
       NOW() AS Started_At;
```

Statistics 1

Name	Value
Queries	118
Updated Rows	565
Execute time	3.4s
Fetch time	0.0s
Total time	3.4s
Start time	2025-10-24 11:05:53.966
Finish time	2025-10-24 11:05:57.826

Script-2

SQL Editor

```
-- TEST CASE 1: BOOKING DETAILS WITH USER INFO
-- Multi-table JOIN across 5 tables
-- =====
-- Business Purpose: Show booking details with guest and host information
-- Tables: bookings, guest_profiles, users, properties, host_profiles, booking_status
SELECT
    b.booking_id,
    CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name,
    CONCAT(hu.first_name, ' ', hu.last_name) AS host_name,
    p.title AS property_title,
    b.check_in_date,
    b.check_out_date,
    b.total_amount,
    bs.status_name AS booking_status
FROM bookings b
    INNER JOIN guest_profiles gp ON b.guest_profile_id = gp.guest_profile_id
    INNER JOIN users gu ON gp.user_id = gu.user_id
    INNER JOIN properties p ON b.property_id = p.property_id
    INNER JOIN host_profiles hp ON p.host_id = hp.host_profile_id
    INNER JOIN users hu ON hp.user_id = hu.user_id
    INNER JOIN booking_status bs ON b.booking_status_id = bs.status_id
WHERE bs.status_name = 'confirmed'
ORDER BY b.created_at DESC
LIMIT 10;
```

bookings(+) 1

SQL Editor

Enter a SQL expression to filter results (use Ctrl+Space)

Grid

Text

	123 booking_id	AZ guest_name	AZ host_name	AZ property_title	check_in_date	check_out_date	123 total_amount	Value
1	20	Maria Martinez	Matthew Martin	Amsterdam Canal House	2025-04-15	2025-04-18	375	
2	19	James Rodriguez	Matthew Martin	Mexico City Loft	2025-03-30	2025-04-02	225	
3	18	Emily Davis	Daniel White	Rio Beach House	2025-03-15	2025-03-21	960	
4	17	Robert Miller	Daniel White	São Paulo Apartment	2025-02-25	2025-02-28	165	
5	16	Lisa Garcia	Chris Thomas	Kyoto Traditional House	2025-02-10	2025-02-13	255	
6	15	David Brown	Chris Thomas	Tokyo Studio	2025-01-25	2025-01-27	70	
7	14	Sarah Wilson	Alex Taylor	Melbourne House	2025-01-10	2025-01-14	560	
8	13	Mike Johnson	Alex Taylor	Sydney Harbor View	2024-12-28	2024-12-31	285	
9	12	Jane Smith	James Rodriguez	Toronto House	2024-12-20	2024-12-25	650	
10	11	John Doe	James Rodriguez	Rome Apartment	2024-12-03	2024-12-06	210	

```
-- TEST CASE 2: PROPERTY PERFORMANCE WITH AGGREGATION
-- Multi-table JOIN across 5 tables with aggregation
-- =====
-- Business Purpose: Analyze property performance with bookings and reviews
-- Tables: properties, host_profiles, users, bookings, reviews, booking_status
SELECT
    p.property_id,
    p.title AS property_title,
    CONCAT(u.first_name, ' ', u.last_name) AS host_name,
    COUNT(DISTINCT b.booking_id) AS total_bookings,
    AVG(r.rating) AS average_rating,
    SUM(b.total_amount) AS total_revenue
FROM properties p
    INNER JOIN host_profiles hp ON p.host_id = hp.host_profile_id
    INNER JOIN users u ON hp.user_id = u.user_id
    LEFT JOIN bookings b ON p.property_id = b.property_id
    LEFT JOIN booking_status bs ON b.booking_status_id = bs.status_id
    LEFT JOIN reviews r ON b.booking_id = r.booking_id
WHERE bs.status_name = 'confirmed' OR bs.status_name IS NULL
GROUP BY p.property_id, p.title, u.first_name, u.last_name
HAVING total_bookings > 0
ORDER BY total_revenue DESC
LIMIT 10;
```

SELECT b.booking_id, CONCAT(u.first_name, ' ', u.last_name) AS host_name, p.property_id, p.title, u.first_name, u.last_name, COUNT(DISTINCT b.booking_id) AS total_bookings, AVG(r.rating) AS average_rating, SUM(b.total_amount) AS total_revenue FROM properties p INNER JOIN host_profiles hp ON p.host_id = hp.host_profile_id INNER JOIN users u ON hp.user_id = u.user_id LEFT JOIN bookings b ON p.property_id = b.property_id LEFT JOIN booking_status bs ON b.booking_status_id = bs.status_id LEFT JOIN reviews r ON b.booking_id = r.booking_id WHERE bs.status_name = 'confirmed' OR bs.status_name IS NULL GROUP BY p.property_id, p.title, u.first_name, u.last_name HAVING total_bookings > 0 ORDER BY total_revenue DESC LIMIT 10;

	123 property_id	AZ property_title	AZ host_name	123 total_bookings	123 average_rating	123 total_revenue
1	3	Luxury Hamptons Villa	John Doe	1	4.5	7,000
2	8	Tuscany Villa	David Brown	1	4.5	4,200
3	2	Cozy Brooklyn House	John Doe	1	4.5	1,200
4	18	Rio Beach House	Daniel White	1	[NULL]	960
5	1	Modern Downtown Apartment	John Doe	1	4.5	900
6	7	Parisian Apartment	David Brown	1	4.5	720
7	10	Barcelona Townhouse	Robert Miller	1	4.5	660
8	12	Toronto House	James Rodriguez	1	[NULL]	650
9	5	Traditional English Cottage	Mike Johnson	1	4.5	600
10	14	Melbourne House	Alex Taylor	1	[NULL]	560


```
-- TEST CASE 3: GUEST BOOKING ANALYSIS WITH WINDOW FUNCTIONS
-- Multi-table JOIN across 5 tables with window functions
-- =====
-- Business Purpose: Analyze guest booking patterns with ranking
-- Tables: users, guest_profiles, bookings, properties, booking_status
SELECT
    gu.user_id,
    CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name,
    COUNT(DISTINCT b.booking_id) AS total_bookings,
    SUM(b.total_amount) AS total_spent,
    ROW_NUMBER() OVER (ORDER BY COUNT(DISTINCT b.booking_id) DESC) AS booking_rank,
    RANK() OVER (ORDER BY SUM(b.total_amount) DESC) AS spending_rank
FROM users gu
    INNER JOIN guest_profiles gp ON gu.user_id = gp.user_id
    INNER JOIN bookings b ON gp.guest_profile_id = b.guest_profile_id
    INNER JOIN properties p ON b.property_id = p.property_id
    INNER JOIN booking_status bs ON b.booking_status_id = bs.status_id
WHERE bs.status_name = 'confirmed'
GROUP BY gu.user_id, gu.first_name, gu.last_name
HAVING total_bookings >= 2
ORDER BY total_bookings DESC
LIMIT 10;
```

SELECT p.property_id, p.title AS property_title, CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name, COUNT(DISTINCT b.booking_id) AS total_bookings, SUM(b.total_amount) AS total_spent, ROW_NUMBER() OVER (ORDER BY COUNT(DISTINCT b.booking_id) DESC) AS booking_rank, RANK() OVER (ORDER BY SUM(b.total_amount) DESC) AS spending_rank

	123 user_id	AZ guest_name	123 total_bookings	123 total_spent	123 booking_rank	123 spending_rank
1	3	Mike Johnson	2	3,785	3	1
2	8	Emily Davis	2	3,060	8	2
3	2	Jane Smith	2	1,250	2	3
4	4	Sarah Wilson	2	800	4	4
5	10	Maria Martinez	2	705	10	5
6	1	John Doe	2	660	1	6
7	7	Robert Miller	2	525	7	7
8	6	Lisa Garcia	2	375	6	8
9	5	David Brown	2	370	5	9
10	9	James Rodriguez	2	360	9	10

*<localhost 3> Script-2 X

```
-- TEST CASE 4: HOST PERFORMANCE ANALYSIS
-- Multi-table JOIN across 5 tables with aggregation
-- =====
-- Business Purpose: Analyze host performance with properties and bookings
-- Tables: users, host_profiles, properties, bookings, reviews, booking_status
SELECT
    hu.user_id,
    CONCAT(hu.first_name, ' ', hu.last_name) AS host_name,
    COUNT(DISTINCT p.property_id) AS total_properties,
    COUNT(DISTINCT b.booking_id) AS total_bookings,
    AVG(r.rating) AS avg_rating,
    SUM(b.total_amount) AS total_revenue
FROM users hu
    INNER JOIN host_profiles hp ON hu.user_id = hp.user_id
    INNER JOIN properties p ON hp.host_profile_id = p.host_id
    LEFT JOIN bookings b ON p.property_id = b.property_id
    LEFT JOIN booking_status bs ON b.booking_status_id = bs.status_id
    LEFT JOIN reviews r ON b.booking_id = r.booking_id
WHERE bs.status_name = 'confirmed' OR bs.status_name IS NULL
GROUP BY hu.user_id, hu.first_name, hu.last_name
HAVING total_properties > 0
ORDER BY total_revenue DESC
LIMIT 10;
```

users 1 X

SELECT hu.user_id, CONCAT(hu.first_name, ' ', hu.last_name) AS host_name, COUNT(DISTINCT p.property_id) AS total_properties, COUNT(DISTINCT b.booking_id) AS total_bookings, AVG(r.rating) AS avg_rating, SUM(b.total_amount) AS total_revenue

Grid		123 user_id	A-Z host_name	123 total_properties	123 total_bookings	123 avg_rating	123 total_revenue
1		1	John Doe	3	3	4.5	9,100
2		5	David Brown	3	3	4.5	5,160
3		16	Daniel White	2	2	[NULL]	1,125
4		3	Mike Johnson	2	2	3.75	1,080
5		7	Robert Miller	2	2	3.75	930
6		9	James Rodriguez	2	2	[NULL]	860
7		12	Alex Taylor	2	2	[NULL]	845
8		18	Matthew Martin	2	2	[NULL]	600
9		14	Chris Thomas	2	2	[NULL]	325

*<localhost 3> Script-2 ✕

```
-- TEST CASE 5: LOCATION-BASED PROPERTY ANALYSIS
-- Multi-table JOIN across 6 tables with subquery
-- =====
-- Business Purpose: Analyze properties by location with market metrics
-- Tables: countries, cities, addresses, properties, bookings, reviews, booking_status, property_pricing
SELECT
    co.country_name,
    c.city_name,
    COUNT(DISTINCT p.property_id) AS total_properties,
    AVG(pp.base_price_per_night) AS avg_price_per_night,
    AVG(r.rating) AS avg_rating,
    SUM(b.total_amount) AS total_revenue
FROM countries co
    INNER JOIN cities c ON co.country_id = c.country_id
    INNER JOIN addresses ad ON c.city_id = ad.city_id
    INNER JOIN properties p ON ad.address_id = p.address_id
    LEFT JOIN property_pricing pp ON p.property_id = pp.property_id
    LEFT JOIN bookings b ON p.property_id = b.property_id
    LEFT JOIN booking_status bs ON b.booking_status_id = bs.status_id
    LEFT JOIN reviews r ON b.booking_id = r.booking_id
WHERE bs.status_name = 'confirmed' OR bs.status_name IS NULL
GROUP BY co.country_id, co.country_name, c.city_id, c.city_name
HAVING total_properties > 0
ORDER BY total_revenue DESC
LIMIT 10;
```

 countries(+) 1 

Grid			 country_name	 city_name	123 total_properties	123 avg_price_per_night	123 avg_rating	123 total_revenue
Text	1		United Kingdom	London	1	500	4.5	7,000
	2		Canada	Toronto	1	300	4.5	4,200
	3		United States	Los Angeles	1	120	4.5	1,200
	4		Finland	Helsinki	1	160	[NULL]	960
	5		United States	New York	1	150	4.5	900
	6		Italy	Rome	1	90	4.5	720
	7		Japan	Tokyo	1	110	4.5	660
	8		Mexico	Mexico City	1	130	[NULL]	650
	9		France	Paris	1	100	4.5	600
	10		Switzerland	Zurich	1	140	[NULL]	560


```
-- TEST CASE 6: MESSAGE CONVERSATION ANALYSIS
-- Multi-table JOIN across 5 tables with aggregation
-- =====
-- Business Purpose: Analyze conversation patterns between hosts and guests
-- Tables: conversations, users, messages, properties, bookings
SELECT
    c.conversation_id,
    CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name,
    CONCAT(hu.first_name, ' ', hu.last_name) AS host_name,
    p.title AS property_title,
    COUNT(m.message_id) AS total_messages,
    MIN(m.sent_at) AS first_message,
    MAX(m.sent_at) AS last_message
FROM conversations c
    INNER JOIN users gu ON c.participant1_id = gu.user_id
    INNER JOIN users hu ON c.participant2_id = hu.user_id
    INNER JOIN bookings b ON c.booking_id = b.booking_id
    INNER JOIN properties p ON b.property_id = p.property_id
    LEFT JOIN messages m ON c.conversation_id = m.conversation_id
GROUP BY c.conversation_id, gu.first_name, gu.last_name, hu.first_name, hu.last_name, p.title
HAVING total_messages > 0
ORDER BY total_messages DESC
LIMIT 10;
```

		123 conversation_id	A-Z guest_name	A-Z host_name	A-Z property_title	123 total_messages	🕒 first_message	🕒 last_message
Text	1	1	John Doe	John Doe	Modern Downtown Apartment	3	2024-01-15 10:30:00	2024-01-15 10:40:00
	2	2	Jane Smith	Jane Smith	Cozy Brooklyn House	2	2024-02-20 14:45:00	2024-02-20 14:50:00
	3	3	Mike Johnson	Mike Johnson	Luxury Hamptons Villa	2	2024-03-10 09:15:00	2024-03-10 09:20:00
	4	4	Sarah Wilson	Sarah Wilson	Historic London Flat	2	2024-04-05 16:20:00	2024-04-05 16:25:00
	5	5	David Brown	David Brown	Traditional English Cottage	2	2024-05-18 11:30:00	2024-05-18 11:35:00

 * <localhost 3> Script-2 ✕

```
-- TEST CASE 7: PAYMENT ANALYSIS WITH SUBQUERY
-- Multi-table JOIN across 4 tables with subquery
-- =====
-- Business Purpose: Analyze payment patterns and amounts
-- Tables: payments, bookings, users, payment_methods
SELECT
    p.payment_id,
    CONCAT(gu.first_name, ' ', gu.last_name) AS guest_name,
    b.booking_id,
    p.amount,
    p.payment_status,
    pm.method_type AS payment_method_type,
    p.processed_at AS payment_date
FROM payments p
    INNER JOIN bookings b ON p.booking_id = b.booking_id
    INNER JOIN guest_profiles gp ON b.guest_profile_id = gp.guest_profile_id
    INNER JOIN users gu ON gp.user_id = gu.user_id
    INNER JOIN payment_methods pm ON p.payment_method_id = pm.payment_method_id
WHERE p.payment_status = 'completed'
    AND p.amount > (SELECT AVG(amount) FROM payments WHERE payment_status = 'completed')
ORDER BY p.amount DESC
LIMIT 10;
```

payments(+) 1 ✕

Grid		123 payment_id	A-Z guest_name	123 booking_id	123 amount	A-Z payment_status	A-Z payment_method_type	🕒 payment_date
Text	1	3	Mike Johnson	3	3,500	completed	bank_transfer	2024-03-10 10:00:00
	2	8	Emily Davis	8	2,100	completed	credit_card	2024-08-14 16:00:00
	3	18	Emily Davis	18	960	completed	credit_card	2025-03-15 16:00:00
	4	12	Jane Smith	12	650	completed	credit_card	2024-12-03 15:00:00

*<localhost 3> Script-2

```
-- TEST CASE 8: AMENITY ANALYSIS WITH AGGREGATION
-- Multi-table JOIN across 4 tables with aggregation
-- =====
-- Business Purpose: Analyze which amenities correlate with higher ratings
-- Tables: property_amenities, property_amenity_links, properties, reviews
SELECT
    pa.amenity_name,
    pa.amenity_category,
    COUNT(DISTINCT pal.property_id) AS properties_with_amenity,
    AVG(r.rating) AS avg_rating,
    COUNT(DISTINCT r.review_id) AS total_reviews
FROM property_amenities pa
INNER JOIN property_amenity_links pal ON pa.amenity_id = pal.amenity_id
INNER JOIN properties p ON pal.property_id = p.property_id
LEFT JOIN bookings b ON p.property_id = b.property_id
LEFT JOIN reviews r ON b.booking_id = r.booking_id
GROUP BY pa.amenity_id, pa.amenity_name, pa.amenity_category
HAVING properties_with_amenity > 1
ORDER BY avg_rating DESC
LIMIT 10;
```

property_amenities 1

SELECT p.payment_id, CONCAT(gu.first_name, ' ', gu.last_name) AS Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z amenity_name	A-Z amenity_category	123 properties_with_amenity	123 avg_rating	123 total_reviews
1	WiFi	Internet	5	4.2	10
2	Kitchen	Kitchen	5	4.2	10
3	Parking	Parking	5	4.2	10
4	Pool	Pool	5	4.2	10
5	Gym	Gym	5	4.2	10

*<localhost 3> Script-2

```
-- VERIFICATION QUERIES
-- =====

-- Check total counts for verification
SELECT 'Users' AS table_name, COUNT(*) AS record_count FROM users
UNION ALL
SELECT 'Properties', COUNT(*) FROM properties
UNION ALL
SELECT 'Bookings', COUNT(*) FROM bookings
UNION ALL
SELECT 'Reviews', COUNT(*) FROM reviews
UNION ALL
SELECT 'Messages', COUNT(*) FROM messages
UNION ALL
SELECT 'Payments', COUNT(*) FROM payments;
```

Results 1

SELECT 'Users' AS table_name, COUNT(*) AS record_count FROM L | Enter a SQL expression to filter results (use Ctrl+Space)

Grid		A-Z table_name	123 record_count	
Text	1	Users	25	
	2	Properties	20	
	3	Bookings	20	
	4	Reviews	20	
	5	Messages	11	
	6	Payments	20	

*<localhost 3> Script-2

```
-- VERIFICATION QUERIES
-- =====

-- Check foreign key relationships
SELECT
    'Foreign Key Check' AS test_type,
    COUNT(*) AS total_records,
    COUNT(DISTINCT guest_profile_id) AS unique_guests,
    COUNT(DISTINCT property_id) AS unique_properties
FROM bookings
WHERE guest_profile_id IS NOT NULL AND property_id IS NOT NULL;
```

Results 1

SELECT 'Users' AS table_name, COUNT(*) AS record_count FROM L | Enter a SQL expression to filter results (use Ctrl+Space)

Grid		A-Z test_type ▼	123 total_records ▼	123 unique_guests ▼	123 unique_properties ▼
	1	Foreign Key Check	20	10	20

